Why Generate a Sentence? A Decision Language Model for Modeling Single-Cell Composition

Anonymous submission

Abstract

Predicting the transcriptome of cells in unmeasured conditions, such as unseen cell types or genetic perturbations, is central to building virtual cells, and language models promise to condition these predictions on text. Current single-cell language models, however, write each cell as a sentence of gene names ranked by expression and generate it token by token: their output must be heuristically mapped back to counts, lacks a count likelihood, and is slow to decode. We introduce CellAsDistribution, which represents a cell on a fixed gene panel as the probability distribution of its transcript counts. A softmax decision head over the panel on a condition encoder returns this distribution from a text prompt in one forward pass, scoring every method by the exact multinomial likelihood of held-out counts. On held-out CRISPRa perturbations in K562, CellAsDistribution outperforms Cell2Sentence fine-tuned on the same backbone by 0.10 nats per UMI, raises correlation on downstream differentially expressed genes from 0.56 to 0.77, decodes 90 to 540 times faster, and matches additive composition of known effects. Pre-registered controls show that this gain comes from the distribution output rather than language-model knowledge, and that no learned representation predicts responses to genes never perturbed in training; on human PBMC, a backbone pretrained on cell sentences generates unseen cell types from their names. By placing sentence models, count models, and simple baselines on one proper scale, our decision-native interface separates how a cell is represented from what its encoder knows.

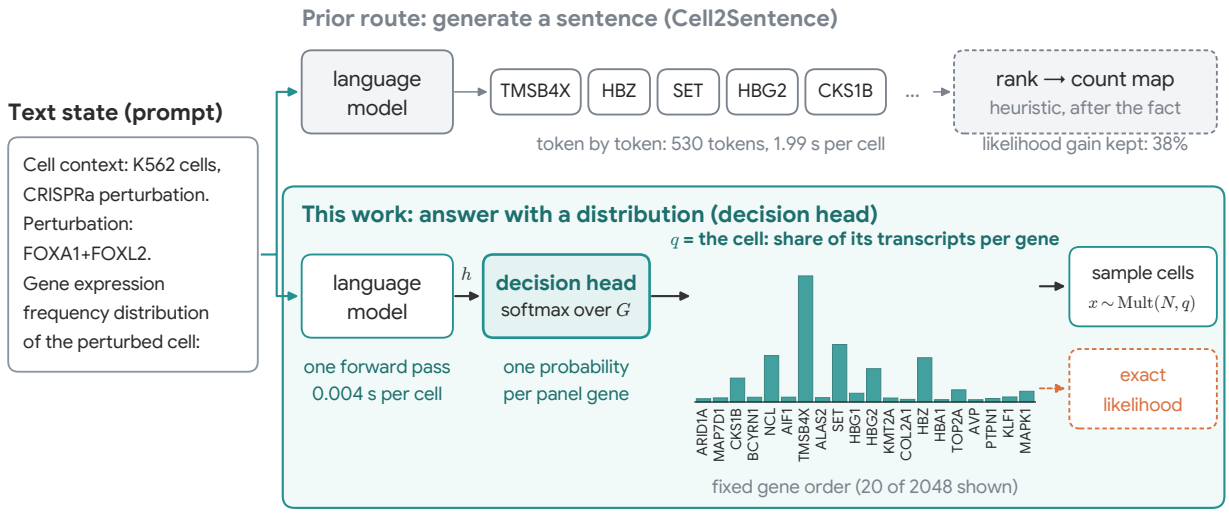


Figure 1: **Why generate a sentence? The same prompt, answered two ways.** Top (prior route): a cell-sentence model writes the cell as gene names ranked by expression, token by token, and a heuristic map turns the ranks back into expression; on our test cells that map keeps 38% of the achievable likelihood gain (Section 5.1). Bottom (this work): a decision head on the language model's last-token state h answers in one forward pass with one probability for every gene of a fixed, ordered panel. That distribution q is the cell: it is sampled to generate cells and scored by the exact multinomial likelihood of held-out counts. Bars: the distribution decoded for the test perturbation FOXA1+FOXL2 on 20 of the 2048 panel genes (Figure 3); timings measured on one A100 (Section 5.1). Teal: conditioning and our model; orange: evaluation.

1 Introduction

Predicting the transcriptional state of cells in an unmeasured condition, such as a cell type in a new donor or a genetic perturbation, is the central task of the "virtual cell" program [1]. A useful generator must say not only which genes change but how a cell's transcripts are distributed over genes, and it must be judged by the probability it assigns to cells it has not seen. Language models are attractive for this task because a condition can be stated in text: Cell2Sentence (C2S) writes each cell as a sentence of gene names ordered by decreasing expression and fine-tunes a pretrained language model on these sentences [2, 3]. The model then generates a cell token by token, and its output is text: a ranked list of names that must be mapped back to expression before it can be compared with counts.

We ask a simple question: why generate a sentence at all? On a fixed panel of G genes, the counts of a cell divided by their sum form a probability distribution over the panel, and together with the library size they reproduce the counts exactly (Proposition 1). A *decision language model* answers a prompt not with text but with a probability over a fixed set of options in one forward pass [4, 5]. If the options are the genes of the panel, the answer *is* a cell: the expected share of its transcripts that each gene contributes. We call this framework CellAsDistribution (Figure 1). Given a text state (a cell type, or a perturbation with an optional control cell), the model outputs a normalized gene distribution in a single pass; counts are sampled from this distribution, and the multinomial likelihood of held-out counts scores the model on the same scale as any other generator. Because the output space is fixed, the condition encoder can be swapped: the same head can sit on a fine-tuned language model or on plain gene embeddings, allowing us to isolate what each component contributes.

Because this formulation is simple, its value must be established empirically, and we use it to dissect text-based cell generation along three questions. **Interface (Section 5.1).** Does generating the distribution directly beat generating a rank sentence on the same backbone and budget, and where does the sentence lose likelihood? **Language model (Section 5.2).** Does the fine-tuned language model matter, or would any encoder of the condition suffice? **Biology (Sections 5.3–5.5).** Does a better gene distribution yield better biology: the downstream response to a perturbation once the trivially induced target gene is set aside, the realism of single cells, and generalization to conditions never seen in training? We answer these questions on human PBMC (generation for unseen donors and unseen cell types) and on the CRISPRa screen of Norman et al. in K562 (response to held-out perturbations), where simple baselines are

known to be hard to beat [6, 7]. Our contributions are:

1. **A decision-native framework for single cells.** A softmax decision head over a fixed gene panel, on a fine-tuned language model or on gene embeddings, returns a normalized gene composition in one pass with an exact multinomial score, placing sentence models, count models, and simple baselines on one proper scale (Section 3).

2. **Where sentence generation loses.** On the same backbone and budget, direct decoding gains 0.10 nats per UMI over autoregressive C2S at 90 to 540 times lower decoding cost; rewriting real cells separates the loss of the published rank-to-count map from the loss of the ranks themselves, and re-decoding generated sentences shows that generation, not only the map, falls short (Section 5.1).

3. **What the language model contributes, and the biological boundary.** Pre-registered controls show that the gain comes from the distribution output and that the language model's measured role is the text interface; under leakage-controlled splits, CellAsDistribution matches additive composition of known effects on likelihood and on downstream differentially expressed genes, shrinks the induction of the targeted gene, generates unseen cell types from their names, and makes no gene-specific prediction for genes never perturbed in training (Sections 5.2–5.5).

2 Related work

Decision language models. A decision language model returns a probability over a bounded set of options for a typed question in a single pass instead of generating text to be parsed [5, 8]. The open implementation we build on attaches a classification head to the final hidden state of a pretrained model and trains it with the cross-entropy on soft targets [4]. The head itself is a standard softmax classifier; what is new here is choosing the genes of a panel as the options, so that the output distribution is a cell.

Cells as text and cell foundation models. Cell2Sentence fine-tunes language models on rank-ordered gene sentences and has been scaled to billions of parameters [2, 3]; its case for the representation is that expression is recovered from rank by a log-linear fit with high R^2. Other foundation models pretrain on expression tokens (scGPT, Geneformer) [9, 10], learn cell embeddings (UCE) [11], or decode the whole profile from a bottleneck token (BMFM-RNA) [12]; none returns a normalized distribution from a text condition.

Count generators and perturbation predictors. scVI and scANVI decode gene frequencies inside a negative-binomial likelihood conditioned on latent states and labels [13, 14]. Our output head uses the same parameterization (a softmax over genes, scaled by library size, optionally with gene-wise dispersion); what differs is its input, a text state read in one pass, which places sentence models, count models, and baselines on one common scale. scDiffusion samples cells with a conditional diffusion model [15], and the Gene Mover's Distance compares cells as distributions over genes [16]. Perturbation predictors such as scGen, CPA, GEARS, and State map a perturbation to an expression change [17–20], and population models learn copulas, flows, or transport between control and perturbed cells [21–26]. Independent benchmarks find that mean, additive, and linear baselines are hard to beat [6, 7] and that control-referenced metrics are inflated by systematic variation [27]; our main references are therefore the perturbed mean and additive composition.

3 Decoding a cell as a gene distribution

3.1 A cell as a distribution over a fixed gene panel

Let $x \in \mathbb{N}^G$ be the raw counts of one cell over a fixed panel of G genes, with library size $N = \sum_g x_g$, and let c be a text *state*: a cell type, or a perturbation together with an optional control cell x_0. We use $G{=}2048$ genes selected on training cells only (largest dispersion of log-normalized expression among genes detected in more than 1% of training cells; for Norman, the training perturbation targets are always included). The panels contain no ribosomal-protein or mitochondrial genes.

Proposition 1 (Lossless representation and proper scoring). *Let $N > 0$ and $p = x/N$. (i) $x \mapsto (N, p)$ is injective, so frequency and library size are lossless, and under $x \sim \mathrm{Mult}(N, \pi)$ the log-likelihood of a model q is $N \sum_g p_g \log q_g +$ const. (ii) If the cells of a unit have expected composition π, the expected NLL per UMI of q is $H(\pi) + \mathrm{KL}(\pi \,\|\, q)$, uniquely minimized at $q = \pi$, so differences in NLL between two decoders are differences in KL divergence to the true composition. (iii) A rank sentence is constant on all count vectors with the same ordering, and log1p or square-root vectors renormalized onto the simplex differ from p unless all nonzero counts are equal: neither keeps what (i) and (ii) score.*

The proposition is elementary; we state it because it fixes what a generator of counts should output and how every method can be scored (proof in Appendix L). We distinguish four objects. The decoder outputs $q(c) \in \Delta^{G-1}$, the *composition* of a cell in state c: $q_g(c)$ is the probability that a transcript of the cell, drawn at random from its panel UMIs, comes from gene g. It is not the probability that g is differentially expressed or that an intervention succeeds. Counts are drawn as $x \mid N, q \sim \mathrm{Mult}(N, q)$. A decoder with a latent variable z outputs $q(c, z)$, and the population of cells in state c, $P(x \mid c)$, is then a mixture. A good fit of $q(c)$ to the mean composition does not by itself make $P(x \mid c)$ realistic, so we evaluate both.

3.2 A decision head on a condition encoder

The state is written as a prompt and read by C2S-Scale-Gemma-2-2B [28, 29], a Gemma-2 model already trained on cell sentences, with rank-16 LoRA adapters on all linear layers [30]. The last-token hidden state h feeds a linear decision head with one logit per panel gene in a fixed panel order, so the model never orders or ranks genes. Instead, it assigns a probability to every position of the gene list via $q_\theta(c) = \mathrm{softmax}(Wh + \log b)$, where $\log b$ is an optional fixed log-prior (none, the smoothed training mean, or the smoothed control cell x_0, which makes the model a residual on the control cell). The head is initialized at zero, so the model starts as the prior and learns a correction. Training minimizes the cross-entropy $-\sum_g p_g \log q_{\theta,g}(c)$ between the frequency vector $p = x/N$ of a real training cell and the decoded q_θ (the log1p and square-root targets of Proposition 1(iii) are worse on Norman for every metric; Appendix E). For perturbations without a control cell, the prompt names the perturbed genes; with a control cell, each training perturbed cell is paired with a random training control cell, and the control cell enters through $\log b$ (Section 5.4).

Encoders and heads. The head does not depend on the encoder. As controls for the language model, we replace h by the frozen language model's state, or by $h = \mathrm{MLP}(\sum_{g \in c} e_g)$, a sum of gene embeddings e_g (learned, with a zero embedding for a gene never perturbed in training, or frozen external embeddings) with the same head and loss. Two heads keep the single pass and the normalization of q while adding what the flat softmax lacks. A negative-binomial head adds a per-gene inverse dispersion $\log \theta_g(c) = \beta_g + v_g^\top h$ and is trained on the negative-binomial likelihood of the counts with mean $N_i q_g$. A low-rank logistic-normal head draws $z = Wh + \log b + U\,\mathrm{diag}(a(h))\,\epsilon$, $\epsilon \sim \mathcal{N}(0, I_{16})$, so that $q(c, z) = \mathrm{softmax}(z)$ varies from cell to cell along 16 learned gene programs; it is trained with a variational bound and an encoder of the target cell.

What a categorical state can learn. When the state is a cell type, the objective averaged over the training cells of the type is minimized, by Gibbs' inequality, at their equal-weight mean frequency; the pooled score below is minimized at the UMI-weighted mean. A decoder conditioned on a label can therefore at best reproduce a type mean (a UMI-weighted loss did not help on validation; Appendix G).

3.3 Sampling cells and scoring them by likelihood

A cell is generated by drawing a library size from training cells and counts from $\mathrm{Mult}(s, q)$, or from a Dirichlet-multinomial with a concentration fit on training cells to represent overdispersion; every method uses the same samplers. For a held-out unit (a cell type or perturbation) with cells $x_1, \ldots, x_n$, we report $\ell(q) = -\sum_i \sum_g x_{ig} \log q_g / \sum_i N_i$, the multinomial negative log-likelihood (NLL) per UMI up to a model-independent constant; differences between models are exact differences in log-likelihood, and by Proposition 1(ii) they equal differences in KL divergence to the true mean composition. Each perturbation receives the same $M = 50$ held-out control cells for every method. A model that conditions on a control cell gives one q_m per control cell; its *pooled* NLL uses $\bar{q} = \frac{1}{M}\sum_m q_m$ and its *cell* NLL is the mixture $-\sum_i \log \frac{1}{M}\sum_m \exp(\sum_g x_{ig} \log q_{mg}) / \sum_i N_i$, which rewards covering the variation between cells. A negative-binomial likelihood with gene-wise dispersions leaves the ordering of methods unchanged (Appendix C). Any method that outputs gene frequencies can be scored this way; methods with a native likelihood (scVI) are additionally scored with their own samples.

4 Data, baselines and evaluation

Conditional generation. We evaluate conditional generation on human PBMC from Hao et al. [31] (CELLxGENE Census; 161,159 cells across 8 donors and 29 cell types), using donors P1–P5 for training, P6 for validation, and P7–P8 for testing; evaluation units are cell types with at least 30 held-out cells. **Perturbation response.** We evaluate perturbation prediction on the CRISPRa screen of Norman et al. in K562 [32, 33] (111,445 cells across 105 single and 131 paired perturbations). In our gene-level split, held-out single genes occur in no training perturbation (an earlier split leaked through training pairs; an audit found it and all its results were discarded). The 76 test perturbations form three strata: 21 unseen single genes, 29 pairs with one unseen gene, and 26 pairs of two genes seen alone in training.

Baselines. We compare against the global and cell-type means; the control and perturbed means; *partial additive* composition, in which an unseen gene contributes nothing; additive plus *target induction*, which multiplies each unseen target's frequency by one factor fit on training single perturbations (CRISPRa raises its target about 4.8-fold), the strongest simple biology-aware baseline we know; ridge regression; scVI, scANVI, and an scGen-style latent shift; GEARS; and autoregressive C2S on the same backbone and budget, whose sentences are mapped to frequencies by the C2S log-rank regression and mixed with the training mean (weight chosen on validation). All baselines operate in frequency space and share the sampler.

Metrics and statistics. Besides NLL, we report MMD and a real-versus-generated classifier AUC in a PCA space fit on training cells (lower is more realistic), along with two biological measures on pseudobulk profiles $\log(1 + 10^4\bar{q})$. Likelihood weights genes by their mass (TMSB4X alone carries about 16% of panel UMIs in K562), so for perturbations we report the *downstream* response: the top-20 differentially expressed genes of each perturbation (largest $|t|$ of a Welch test between its real cells and held-out controls), *after removing the perturbed genes*, and on them r_{DE}, the Pearson correlation of predicted and real change over the control mean, and the fraction of genes changed in the right direction. The perturbed genes are scored separately as *target error*, the absolute error of their predicted log fold change. Units are cell types or perturbations; intervals are paired bootstraps over units. Decisions were made on validation; each test configuration was evaluated once per seed under promotion rules written down before the test runs, and analyses added after the test results are labeled as such (Appendix G).

5 Results

5.1 One forward pass of CellAsDistribution beats generating a gene-rank sentence

Against Cell2Sentence on the same backbone and budget. We fine-tuned C2S-Scale-Gemma-2-2B autoregressively on rank sentences with the same LoRA rank, training cells, epochs, batch size, and control cells as CellAsDistribution, and decoded its generated sentences with the C2S regression. The sentences are well formed (115 genes on average, 99.8% valid names), yet they add no likelihood beyond the perturbed mean (-0.001 nats per UMI, CI -0.006 to $+0.005$; the validation-chosen mixing weight is the largest on the grid, 0.9). CellAsDistribution is 0.103 nats per UMI better (CI 0.071 to 0.139) and recovers the downstream response far better (r_{DE} 0.77 against 0.56; Table 2). Decoding the same generated sentences with a rank-to-frequency map learned on training cells (described next) barely helps: CellAsDistribution remains 0.098 nats per UMI better (CI 0.069 to 0.131), with the same r_{DE} gap, so the shortfall lies in the generated sentences and not only in the map that decodes them (post hoc).

Decoding cost. Run alone on one A100 GPU at batch size 25, C2S-Gemma requires 1.99 seconds per generated cell (530 new tokens on average), whereas CellAsDistribution requires 0.004 seconds per cell with the control cell as a residual and 0.022 seconds when the control cell is also written as text, a 90- to 540-fold speedup.

Where a sentence loses likelihood. To separate the representation from the model, we rewrote the *real* held-out cells of each unit in a representation, decoded them back to frequencies, and asked how much of the likelihood gain of the cells' own frequency vectors survives (Figure 2). With the published log-rank map, the rank sentence reconstructs log expression well ($R^2 = 0.68$ on Norman, 0.74 on PBMC), yet keeps only 38% of the gain on Norman, against 73% for value bins and 64% for mass levels. A map learned on training cells, the mean frequency of each gene at each rank position, keeps 88% from the same sentences (90% when the library size is also given). Most of the loss therefore lies in the map from ranks to counts rather than in the ranks themselves: the ordering of a K562 cell's genes carries most of its composition, but a sentence must be decoded, and the decoder determines how much survives, with about a tenth lost under every map we tried. On PBMC the representations do not separate (56–58% each), because cell types differ mostly in which genes are expressed. A generator that outputs the composition needs no such map, and its likelihood is defined by the model itself.

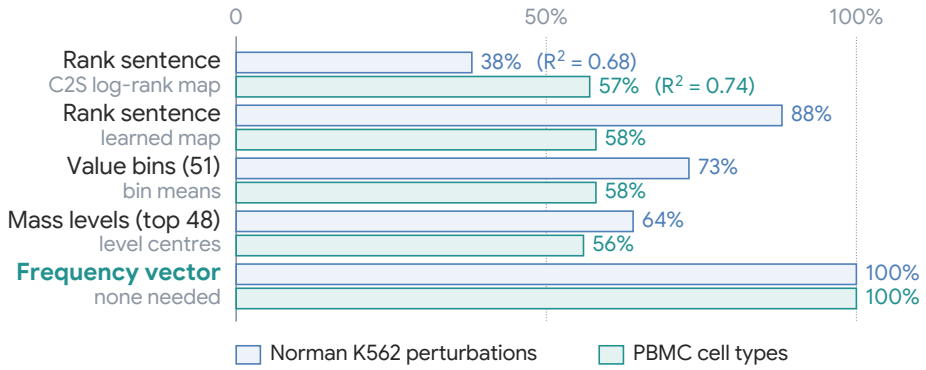


Figure 2: **The likelihood a rank sentence loses depends on how it is decoded.** Real held-out test cells were rewritten in each representation and decoded back to gene frequencies; bars give the share of the likelihood gain of the cells' own frequency vectors over the reference (perturbed mean for Norman, cell-type mean for PBMC) that survives, all estimated from the same cells in the same way. The C2S log-rank map keeps 38% on Norman although it reconstructs log expression with $R^2 = 0.68$, the criterion used to motivate rank sentences [2]; a map learned on training cells keeps 88%. Shares from decoding real cells, not model results; the learned map was added after the test results.

Model	ΔNLL, all ↓	single ↓	1 unseen ↓	both seen ↓	Δr_{DE} ↑
CellAsDistribution (reference)	0	0	0	0	**0**
No LM, learned	**−0.005 [−.017, +.009]**	**−0.011**	−0.003	**−0.002**	−0.012 [−.042, +.016]
No LM, GenePT	+0.031 [+.007, +.067]	+0.011	+0.056	+0.020	−0.066 [−.098, −.035]
No LM, co-expression	+0.006 [−.008, +.021]	+0.001	+0.009	+0.006	−0.037 [−.075, −.001]
Frozen LM + MLP	+0.043 [+.025, +.060]	+0.008	+0.021	+0.096	−0.108 [−.158, −.061]
Additive	+0.009 [−.010, +.037]	+0.006	**−0.010**	+0.033	−0.004 [−.040, +.033]
Additive + target induction	+0.008 [−.010, +.036]	+0.003	**−0.010**	+0.033	−0.004 [−.040, +.033]

Table 1: Does the language model matter? Pooled NLL per UMI of each model minus that of CellAsDistribution without a control cell (same seed; positive values favor CellAsDistribution), Norman test split, seed 0: overall with [95% bootstrap over perturbations], and the mean difference by stratum (single $n = 21$, 1 unseen $n = 29$, both seen $n = 26$). Δr_{DE}: the same paired difference in downstream r_{DE} (perturbed genes excluded; negative favors CellAsDistribution). Every model uses the same 20,000 training cells, frequency target, training-mean log-prior, softmax over the panel and scoring. Overall NLL difference by seed (learned no LM / frozen LM): seed 0: -0.005 / $+0.043$; seed 1: -0.007 / $+0.044$; seed 2: -0.003 / $+0.047$. GenePT: frozen text embeddings of NCBI gene summaries [34]; co-expression: loadings of training cells' principal components; both give every perturbed gene, including unseen ones, an embedding (post hoc). Pre-registered (Appendix G). Differences whose interval covers zero are ties. Best value of each column in bold; arrows give the better direction.

5.2 The gain comes from the distribution output, not from the language model

Matched controls. We compared CellAsDistribution without a control cell against two controls, specified before they were run (Table 1), with the same training cells, target, log-prior, softmax output, and scoring: the same language model *frozen*, with only an MLP head trained on its last-token state; and a decoder with *no language model*, in which a perturbation is the sum of learned embeddings of its genes (an unseen gene gets a zero embedding) followed by an MLP. On test, the decoder without a language model ties CellAsDistribution in every stratum and for every seed (overall -0.005 nats per UMI, CI -0.017 to $+0.009$; seeds 1 and 2: -0.007 and -0.003) and on the downstream response (r_{DE} -0.012, CI -0.042 to $+0.016$), with 1.1 million parameters trained in 10 seconds. The frozen language model is worse ($+0.043$, CI $+0.025$ to $+0.060$; r_{DE} -0.108), most on pairs of seen genes ($+0.096$). On validation, with the Qwen backbone, the no-language-model decoder was 0.013 nats worse (CI 0.000 to 0.028); by the rule fixed before these runs, which is evaluated on test, the likelihood gain over C2S and the perturbed mean comes from decoding the distribution directly, not from knowledge in the language model.

Model	ΔNLL to additive ↓	cell NLL ↓	MMD ↓	AUC ↓	r_{DE} ↑	sign ↑
Perturbed mean	+0.094 [+.058, +.136]	5.289	0.073	0.903	0.58	0.81
Additive (reference)	0	5.195	0.052	0.855	**0.77**	**0.90**
Additive + target induction	−0.001 [−.003, +.000]	5.194	0.051	0.857	**0.77**	**0.90**
scGen-style	+0.006 [−.014, +.022]	5.105	**0.029**	**0.769**	0.72	0.85
C2S, autoregressive	+0.094 [+.059, +.133]	5.279	0.075	0.906	0.56	0.81
Additive, control residual	+0.005 [−.001, +.011]	5.134	0.038	0.815	0.76	0.89
CellAsDistribution, no control cell	**−0.009 [−.037, +.010]**	5.186	0.039	0.812	**0.77**	0.89
CellAsDistribution, control residual	+0.002 [−.025, +.022]	5.129	0.030	0.785	0.72	0.86
CellAsDistribution, NB head	+0.006 [−.021, +.028]	5.201	0.036	0.797	**0.77**	0.88
CellAsDistribution, low-rank head	+0.053 [+.024, +.084]	**5.096**	0.040	0.791	0.73	0.87

Table 2: Perturbation response, Norman K562 test split (76 held-out perturbations, matched control cells, seed 0, C2S-Scale-Gemma-2-2B for every language model). ΔNLL: pooled NLL per UMI minus the additive baseline's [95% bootstrap over perturbations; the additive baseline's NLL is 5.195]. cell NLL: mixture over the $M = 50$ control cells (or the low-rank head's draws). MMD and real-versus-generated AUC on Dirichlet-multinomial samples. Downstream biology, with the perturbed genes removed (added after the test results; post hoc): r_{DE}, Pearson correlation of predicted and real pseudobulk change over the held-out control mean on the perturbation's top-20 differentially expressed genes; sign, the fraction of those genes whose change has the right sign. Best value of each column in bold; arrows give the better direction.

Biological gene embeddings. Giving every gene, including unseen ones, a frozen biological embedding does not help either (post hoc). GenePT embeddings of NCBI gene summaries [34] are worse than learned embeddings overall ($+0.031$ against CellAsDistribution, CI $+0.007$ to $+0.067$) and on unseen single genes ($+0.022$ against learned embeddings, CI $+0.005$ to $+0.040$), and co-expression embeddings from training cells tie CellAsDistribution ($+0.006$, CI -0.008 to $+0.021$) with a lower r_{DE}.

Gene names and backbone. Fine-tuning is what makes the language model as good a decoder of the condition as a purpose-built encoder; its added value is the interface, one prompt format for any state, which lets the same model generate cell types it has never seen (Section 5.5). Replacing gene symbols by opaque identifiers worsens the downstream response even on pairs of seen genes (r_{DE} 0.87 against 0.89; overall 0.69 against 0.77). Since an identity-only embedding without any text matches the model that reads real names, the names act as identifiers, and opaque tokens that share sub-words (GENE001, GENE002, ...) blur them; on unseen single genes the names bring nothing measurable ($+0.007$ nats, CI -0.022 to $+0.030$), nor do NCBI gene summaries (validation). The backbone matters modestly: across three seeds, the C2S-initialized Gemma is 0.011 to 0.012 nats better than the general-purpose Qwen3.5-0.8B on both tasks, more than the seed spread (0.001 to 0.003), without changing any ranking; LoRA ranks 8 to 64 give the same NLL.

5.3 CellAsDistribution matches additive composition downstream but shrinks target induction

Likelihood and realism (Table 2). Without a control cell, CellAsDistribution is 0.104 nats per UMI better than the perturbed mean (CI 0.071 to 0.139) and ties the additive baseline (-0.009, CI -0.037 to $+0.010$). Its generated cells are closer to real ones than additive cells (MMD 0.039 against 0.052; AUC 0.81 against 0.86), but not than those of the scGen-style shift, which moves each control cell individually. GEARS with default settings is 0.47 nats above the perturbed mean after its regression output is mapped to frequencies (Appendix K); this measures that conversion as much as the model.

Downstream response, target gene set aside (Table 3; post hoc). CRISPRa raises its own target, so a model can look right on the target alone. With the perturbed genes removed, CellAsDistribution reproduces the direction of 89% of the top-20 downstream genes and ties additive on r_{DE} overall (0.77 against 0.77; $+0.004$, CI -0.033 to $+0.040$) and in every stratum, and is better than scGen-style (0.72) and C2S (0.56). On the target genes themselves, it is *worse* than additive on pairs (target error 0.51 against 0.27 with one unseen gene, 0.58 against 0.39 with both seen). The error is directional: the true induction of a target is about 1.7 log units, additive composition copies it from the training single perturbation almost without bias ($+0.04$ to $+0.09$), and CellAsDistribution under-predicts it

	ΔNLL to additive ↓			downstream r_{DE} ↑			target error ↓	
Model	single	1 unseen	2 seen	single	1 unseen	2 seen	1 unseen	2 seen
Additive	0	**0**	0	0.51	**0.83**	**0.90**	0.27	0.39
Additive + target induction	−0.003	**+0.000**	−0.000	0.51	**0.83**	**0.90**	**0.24**	0.39
scGen-style	−0.002	+0.037	−0.021	0.44	0.79	0.87	0.74	0.65
No LM (gene embeddings)	**−0.017**	+0.007	**−0.035**	0.52	0.82	0.88	0.40	0.40
CellAsDistribution, no control cell	−0.006	+0.010	−0.033	**0.54**	**0.83**	0.89	0.51	0.58
CellAsDistribution + target prior	−0.009	+0.008	**−0.035**	**0.54**	**0.83**	0.89	0.26	**0.29**
CellAsDistribution, NB head	−0.016	+0.029	−0.002	**0.54**	**0.83**	0.89	0.63	0.67
CellAsDistribution, opaque gene ids	+0.001	+0.029	−0.019	0.42	0.73	0.87	0.72	0.76

Table 3: Norman test split by stratum: unseen single genes ($n = 21$), pairs with one unseen gene ($n = 29$) and pairs of two genes seen alone in training ($n = 26$). ΔNLL: mean paired difference to the additive baseline (intervals in the text). Downstream r_{DE} as in Table 2, perturbed genes excluded. Target error: mean absolute error of the predicted log fold change of the perturbed gene(s) themselves over the held-out control mean, for targets in the panel. Target prior: every target seen alone in training gets its own training-single frequency, unseen targets the global induction factor, applied to the saved CellAsDistribution output. Added after the test results (post hoc). Best value of each column in bold; arrows give the better direction.

by about 0.5, shrinking a large gene-specific change toward the mean composition it learns a correction to (only 4 of the 21 unseen single targets are in the main panel, so this concerns pairs). The shrinkage is an output effect that a prior removes (post hoc): setting each target seen alone in training to its own training-single frequency, and multiplying unseen targets by the global induction factor, lowers CellAsDistribution's target error to 0.26 and 0.29, at or below additive, without changing its likelihood (-0.011 against additive, CI -0.038 to $+0.007$) or its downstream r_{DE}. The negative-binomial head, which reweights genes by dispersion, does not (target error 0.63 and 0.67). Because a target gene carries little transcript mass, these changes move the NLL by less than 0.003 nats; likelihood and target accuracy are separate properties and we report both.

Genes never perturbed in training. On the 21 genes never perturbed in training, no learned model makes a gene-specific prediction: CellAsDistribution ties the perturbed mean (-0.006, CI -0.025 to $+0.018$; r_{DE} 0.54 against 0.51), and the no-language-model decoder, 0.017 nats better (CI 0.005 to 0.028), gives every unseen gene the same decoded empty state, a generic perturbation response; a split-half oracle lies 0.108 nats below the perturbed mean (Appendix A). To score the targets themselves, we rebuilt the panel with every perturbed gene forced in, keeping the splits (Appendix J; post hoc): on the 19 unseen single genes whose target is then in the panel, CellAsDistribution, the no-language-model decoder, and both embedding variants all predict almost no self-induction (target error 1.6 to 1.7 log units against 1.8 for the perturbed mean; the target goes up in only 26 to 42% of predictions), whereas the one-factor target-induction baseline, which encodes only that CRISPRa raises its target, halves the error (0.93) and gets the direction right for 95% of them.

Worked examples (Figure 3). For the median unseen gene, MAML2, its own induction is the largest change among its top-20 DE genes, and neither CellAsDistribution nor any baseline predicts it: a decoder that has never seen the gene perturbed cannot infer this from its name. For the median pair of seen genes under the control residual, FOXA1+FOXL2, CellAsDistribution reproduces the downstream program in direction (Pearson 0.87 on the top-20 DE genes, against 0.83 for additive) while compressing the largest changes, the same shrinkage as for target genes; and on the same fixed gene axis, each control cell yields its own distribution around the mean, which is how the per-cell model represents cell-to-cell variation.

5.4 Cell-to-cell variation enters through the control cell

A single q per perturbation leaves all cell-to-cell variation to the sampler. With the control cell as the log-prior ($b = \mathrm{smooth}(x_0)$, not written in the prompt), the cell NLL improves from 5.186 to 5.129 and the cells become more realistic (MMD 0.030, AUC 0.79), approaching scGen-style on MMD. Writing the control cell into the prompt instead loses this gain ($+0.019$ nats per UMI in cell NLL, CI $+0.012$ to $+0.027$). A version of the residual without a language model, in which the additive shift replaces the learned correction, reaches the same likelihood (5.134) but produces less

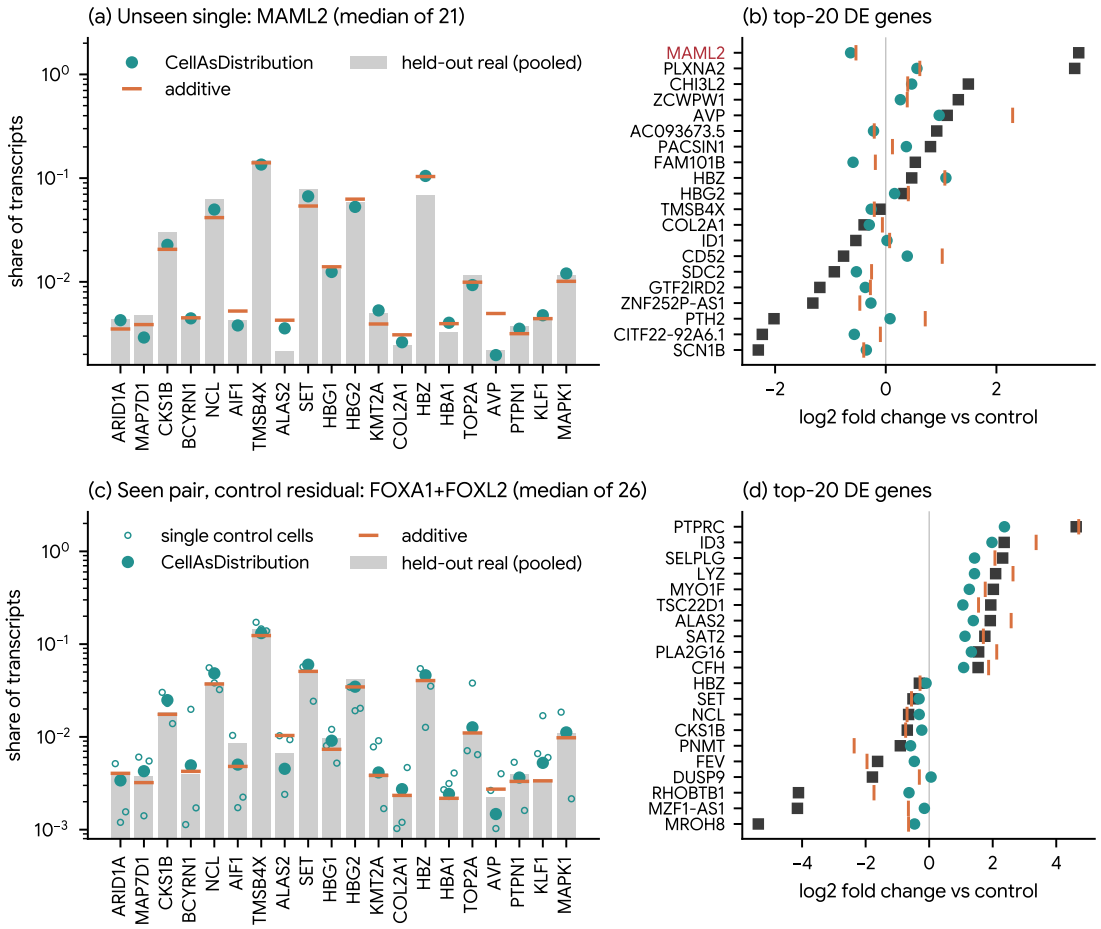


Figure 3: **What the decoder gets right and wrong, on two median test perturbations** (the unit with the median paired NLL difference to additive in its stratum, so not hand-picked; Norman test split, Gemma, seed 0). (a, c) The decoded distribution on a fixed gene axis: the 20 panel genes with the highest mean frequency in training cells, in the order of the decision head's outputs and identical in both rows, so only the distribution changes. Grey: held-out real cells; teal: CellAsDistribution; orange: additive composition; hollow teal: distributions decoded for three individual control cells. (b, d) Log fold change over the held-out control mean of the perturbation's top-20 DE genes (Welch t on real cells), perturbed genes in red. (a, b) MAML2, never perturbed in training: no model predicts its self-induction; CellAsDistribution and additive (here the perturbed mean) are both near the mean composition. (c, d) FOXA1+FOXL2, both genes seen alone in training, with the control cell as the log-prior: CellAsDistribution recovers the direction of the downstream changes but compresses the largest ones, and each control cell gives a different distribution on the same genes.

realistic cells (MMD $+0.008$, AUC $+0.030$ relative to CellAsDistribution). The residual has a cost: its downstream r_{DE} falls to 0.72, below additive (-0.046, CI -0.092 to -0.008), because each control cell's own deviations enter the prediction.

Distributional heads. Two heads keep the single pass and the normalization of q while modeling variation the flat softmax lacks (Appendix H): a low-rank logistic-normal head, $q(c, z)$ along 16 learned gene programs, lowers the cell NLL to 5.096 without any control cell at the cost of the pooled NLL ($+0.062$); a negative-binomial head with per-gene dispersion improves the change correlation over all genes (r_{Δ} $+0.020$) at a small cost in NLL. Gene–gene correlations of generated cells on 100 variable genes are close to those of real cells for every method at this sequencing depth (Appendix I).

Model	ΔNLL to type mean ↓	cell NLL ↓	MMD ↓	AUC ↓	type acc. ↑
Global mean	+1.125 [+.939, +1.326]	6.732	0.833	1.000	0.04
Cell-type mean (reference)	0	5.606	0.182	0.985	**0.88**
scVI	+0.005 [−.007, +.016]	5.611	0.166	0.975	0.82
scVI, own NB sampling	**−0.001 [−.011, +.010]**	5.489	**0.128**	**0.924**	0.70
scANVI, own NB sampling	+0.003 [−.009, +.017]	**5.486**	0.130	0.930	0.67
CellAsDistribution	+0.017 [+.001, +.036]	5.623	0.163	0.970	0.82
CellAsDistribution, low-rank head	+0.041 [+.018, +.067]	5.552	0.165	0.970	0.77

Table 4: Conditional generation, human PBMC test split (unseen donors; 24 cell types; seed 0; Gemma backbone). ΔNLL: paired difference to the cell-type mean [95% bootstrap over cell types]. *Own NB sampling*: cells drawn from scVI/scANVI's own negative binomial from latent states of training cells, with cell NLL as the mixture over these draws; other rows use the shared samplers (generation metrics on Dirichlet-multinomial samples). Best value of each column in bold; arrows give the better direction.

Unseen cell type	ΔNLL to global mean ↓	ΔNLL to name overlap ↓	type acc. ↑
Nearest Cell Ontology type	−0.209 [−.612, +.328]	+0.440 [+.133, +.860]	0.02
Cell Ontology parents	−0.515 [−.737, −.285]	+0.134 [+.001, +.299]	0.05
Name-overlap type (string match)	−0.649 [−.877, −.388]	0	0.02
CellAsDistribution, opaque type id	+0.106 [−.154, +.377]	+0.754 [+.470, +1.041]	0.01
CellAsDistribution, type name	−0.707 [−.875, −.542]	−0.059 [−.216, +.097]	0.15
CellAsDistribution, name + definition	−0.707 [−.862, −.545]	−0.058 [−.219, +.101]	0.14
CellAsDistribution (Gemma), opaque type id	−0.024 [−.322, +.303]	+0.625 [+.321, +.958]	0.04
CellAsDistribution (Gemma), type name	**−0.865 [−1.019, −.707]**	**−0.216 [−.370, −.088]**	**0.20**
CellAsDistribution (Gemma), name + definition	−0.839 [−.992, −.680]	−0.190 [−.344, −.056]	0.18

Table 5: Zero-shot cell types, PBMC test split: 19 types, each held out of training in one of five leave-types-out folds (Qwen3.5-0.8B and, added after the test results, C2S-Scale-Gemma-2-2B backbones; seed 0). Paired differences [95% bootstrap over cell types]. Baselines use training types only. Best value of each column in bold; arrows give the better direction.

5.5 CellAsDistribution reaches the type mean for known cell types and generates unseen ones from their names

Known types (Table 4). As the remark of Section 3.2 predicts, no model improves on the cell-type mean in likelihood on unseen donors: CellAsDistribution is slightly worse ($+0.017$, CI $+0.001$ to $+0.036$), scVI and scANVI tie it, and on validation CellAsDistribution gives the same NLL to four decimals for every LoRA rank and both backbones. Sample realism is decided by the source of cell-to-cell variation: scVI sampling from its own negative binomial reaches the lowest MMD and AUC (0.92), every frequency decoder relies on the shared sampler, and no generator is nearly indistinguishable from real cells of unseen donors (AUC ≥ 0.92).

Unseen types (Table 5). A model indexed by cell type cannot generate a type it has never seen; a decoder that reads the type's name can. In five leave-types-out folds over 19 types, CellAsDistribution with the Qwen backbone generates each held-out type from its name and Cell Ontology definition. It is 0.707 nats per UMI better than the global mean (CI 0.545 to 0.862) and better than the nearest Cell Ontology type and the pooled Cell Ontology parents; it ties a string-matching baseline that copies the training type whose name shares the most words (-0.058, CI -0.219 to $+0.101$) but recovers the requested type far more often (0.14 against 0.02). With the general-purpose Qwen backbone, the definition adds nothing beyond the name, and opaque identifiers remove the gain: that model generalizes by composing the words of type names. With the C2S-Scale-Gemma backbone, pretrained on cell sentences annotated with cell types (post hoc; same folds and settings), the name alone does better than string matching (-0.216, CI -0.370 to -0.088; -0.865 against the global mean) and recovers the requested type more often (0.20); definitions again add nothing, and opaque identifiers again fail. This is the one place in our study where pretrained knowledge helps, and it is knowledge of cell-type names learned from cell data: the pretraining corpus of C2S-Scale contains

annotated immune cells, possibly including these types, so the gain may reflect exposure to them rather than inference from language.

6 Discussion and limitations

Letting a decision language model answer with a gene distribution makes a language model a single-cell generator whose output is a cell's composition rather than a sentence about it. The interface has concrete consequences: generation costs one forward pass, the model's likelihood is defined without a reconstruction map, and every method can be compared on one proper score. The comparison with rank sentences shows why the last point matters: the same sentences keep 38% or 88% of the achievable likelihood depending on the map chosen to decode them, so a sentence model's quality cannot be separated from a post hoc choice, whereas a distribution is scored as produced.

Our controls are equally clear about what the language model does and does not do. On perturbations, a decoder of learned gene embeddings with the same output matches it, and neither a frozen language model nor biological gene embeddings do better, so the decision-native output, not language-model knowledge, carries the gain. For cell types, the language model is what makes unseen types reachable at all, and a backbone pretrained on annotated cell sentences reads their names better than string matching. The interface therefore separates two questions that sentence generation entangles: how a cell is represented and scored, which the distribution head settles, and what prior knowledge the encoder brings, which can then be measured.

A better gene distribution is not yet better biology, and our biological results draw a clear boundary. Where training contains the perturbed genes, the decoder matches additive composition on likelihood and on the downstream program, and generates more realistic cells; it shrinks the target's own induction, which an explicit prior corrects; and for genes never perturbed in training, none of the learned representations we tested (gene names, NCBI summaries, GenePT, or co-expression embeddings) predicts even the self-induction captured by a one-parameter biological prior. A categorical cell type caps any decoder at the type mean, while names let it generate unseen types. The output q is a composition, not a probability of differential expression or of an intervention's success; a decision head trained on such events, with calibration measured on held-out perturbations, is the natural next step for the decision-native interface, together with richer states (donor, cell, and pathway context) and heads that model dispersion and covariance jointly.

Limitations. Our study is limited to one dataset per task, a fixed 2048-gene panel, and CRISPRa perturbations; held-out control cells are not paired with perturbed cells, so cell NLL rewards covering cell states rather than predicting individual counterfactuals; GEARS was evaluated with default settings, and CPA as well as transport and flow models were omitted because they do not output a distribution over a fixed gene list; the C2S-Scale pretraining corpus may contain the PBMC cell types, so the Gemma zero-shot result is not a clean test of generalization; and the downstream, target, target-prior, add-on-panel, gene-embedding, Gemma zero-shot, co-expression, and learned-rank-map analyses were added after the test results (Appendix G).

Code availability. Code, held-out unit lists, run scripts, and table-generation scripts are available with a README at https://github.com/kagtgi/CellAsDistribution.

AI use statement. Code, experiments, tables, figures, and a first draft of this text were produced with assistance from Claude (Anthropic); the authors reviewed all content and are responsible for it.

References

[1] Charlotte Bunne, Yusuf Roohani, Yanay Rosen, et al. How to build the virtual cell with artificial intelligence: Priorities and opportunities. *Cell*, 187:7045–7063, 2024. doi: 10.1016/j.cell.2024.11.015.

[2] Daniel Levine, Syed A. Rizvi, Sacha Lévy, et al. Cell2Sentence: Teaching large language models the language of biology. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 27299–27325, 2024. URL https://proceedings.mlr.press/v235/levine24a.html.

[3] Syed Asad Rizvi, Daniel Levine, Aakash Patel, et al. Scaling large language models for next-generation single-cell analysis. bioRxiv, 2025.

[4] Unsloth. Train your own decision model with Unsloth. Documentation, 2026. URL https://unsloth.ai/docs/basics/train-your-own-decision-model-with-unsloth.

[5] Diogo Almeida. Introducing system one models & Jev. TypeSafe AI blog, 15 September, 2026. URL https://typesafe.ai/blog/introducing-system-one-models-and-jev.

[6] Constantin Ahlmann-Eltze, Wolfgang Huber, and Simon Anders. Deep-learning-based gene perturbation effect prediction does not yet outperform simple linear baselines. *Nature Methods*, 22:1657–1661, 2025. doi: 10.1038/s41592-025-02772-6.

[7] Zhiting Wei, Yiheng Wang, Yicheng Gao, et al. Benchmarking algorithms for generalizable single-cell perturbation response prediction. *Nature Methods*, 23:451–464, 2026. doi: 10.1038/s41592-025-02980-0.

[8] Tobias Deußer, Lorenz Sparrenberg, and Rafet Sifa. Evaluating and benchmarking the system one model Jev. arXiv:2609.37647, 2026.

[9] Haotian Cui et al. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21:1470–1480, 2024. doi: 10.1038/s41592-024-02201-0.

[10] Christina V. Theodoris et al. Transfer learning enables predictions in network biology. *Nature*, 618:616–624, 2023. doi: 10.1038/s41586-023-06139-9.

[11] Yanay Rosen, Yusuf Roohani, Ayush Agrawal, et al. Universal cell embedding provides a foundation model for cell biology. *Nature*, 656(8126):183–191, 2026. doi: 10.1038/s41586-026-10689-z.

[12] Michael M. Danziger, Bharath Dandala, Viatcheslav Gurev, et al. BMFM-RNA: whole-cell expression decoding improves transcriptomic foundation models. arXiv:2506.14861, 2025.

[13] Romain Lopez, Jeffrey Regier, Michael B. Cole, Michael I. Jordan, and Nir Yosef. Deep generative modeling for single-cell transcriptomics. *Nature Methods*, 15:1053–1058, 2018. doi: 10.1038/s41592-018-0229-2.

[14] Chenling Xu, Romain Lopez, Edouard Mehlman, et al. Probabilistic harmonization and annotation of single-cell transcriptomics data with deep generative models. *Molecular Systems Biology*, 17(1):e9620, 2021. doi: 10.15252/msb.20209620.

[15] Erpai Luo, Minsheng Hao, Lei Wei, et al. scDiffusion: conditional generation of high-quality single-cell data using diffusion model. *Bioinformatics*, 40(9):btae518, 2024. doi: 10.1093/bioinformatics/btae518.

[16] Riccardo Bellazzi, Andrea Codegoni, Stefano Gualandi, Giovanna Nicora, and Eleonora Vercesi. The Gene Mover's Distance: Single-cell similarity via optimal transport. arXiv:2102.01218, 2021.

[17] Mohammad Lotfollahi, F. Alexander Wolf, and Fabian J. Theis. scGen predicts single-cell perturbation responses. *Nature Methods*, 16:715–721, 2019. doi: 10.1038/s41592-019-0494-8.

[18] Mohammad Lotfollahi, Anna Klimovskaia Susmelj, Carlo De Donno, et al. Predicting cellular responses to complex perturbations in high-throughput screens. *Molecular Systems Biology*, 19(6):e11517, 2023. doi: 10.15252/msb.202211517.

[19] Yusuf Roohani, Kexin Huang, and Jure Leskovec. Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nature Biotechnology*, 42:927–935, 2024. doi: 10.1038/s41587-023-01905-6.

[20] Abhinav K. Adduri, Dhruv Gautam, Beatrice Bevilacqua, et al. Predicting cellular responses to perturbation across diverse contexts with State. *Cell*, 189(19):5914–5931.e20, 2026. doi: 10.1016/j.cell.2026.07.052.

[21] Handong Wang, Jiaxin Qi, Haochen Feng, and Baisheng Lai. PopPert: Population-level joint-distribution modeling for single-cell perturbation prediction. arXiv:2609.01357, 2026.

[22] Chenglei Yu, Chuanrui Wang, Bangyan Liao, and Tailin Wu. scDFM: Distributional flow matching model for robust single-cell perturbation prediction. In *International Conference on Learning Representations*, 2026. arXiv:2602.07103.

[23] Shuizhou Chen, Lang Yu, Xueqin Lin, et al. SCALE: Scalable conditional atlas-level endpoint transport for virtual cell perturbation prediction. arXiv:2603.17380, 2026.

[24] Bowen Han, Lingbei Meng, Shihuan Luo, et al. TempoBridge: Source-conditioned flow matching with optimal transport couplings for single-cell population transitions. arXiv:2610.04945, 2026.

[25] Changxi Chi, Yufei Huang, Jun Xia, et al. Departures: Distributional transport for single-cell perturbation prediction with neural Schrödinger bridges. arXiv:2511.13124, 2025.

[26] Danning Jiang, Zhiwen Yan, Qirun Wang, et al. OCOO-T: A simple and scalable virtual cell model for transcriptional perturbation response prediction. arXiv:2606.12838, 2026.

[27] Ramon Viñas Torné, Maciej Wiatrak, Zoe Piran, et al. Systema: a framework for evaluating genetic perturbation response prediction beyond systematic variation. *Nature Biotechnology*, 44(6):1050–1059, 2026. doi: 10.1038/s41587-025-02777-8.

[28] van Dijk Lab. C2S-Scale-Gemma-2-2B (model card). Hugging Face, vandijklab/C2S-Scale-Gemma-2-2B, 2025. URL https://huggingface.co/vandijklab/C2S-Scale-Gemma-2-2B.

[29] Gemma Team. Gemma 2: Improving open language models at a practical size. arXiv:2408.00118, 2024.

[30] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. arXiv:2106.09685.

[31] Yuhan Hao et al. Integrated analysis of multimodal single-cell data. *Cell*, 184:3573–3587.e29, 2021. doi: 10.1016/j.cell.2021.04.048.

[32] Thomas M. Norman, Max A. Horlbeck, Joseph M. Replogle, et al. Exploring genetic interaction manifolds constructed from rich single-cell phenotypes. *Science*, 365:786–793, 2019. doi: 10.1126/science.aax4438.

[33] Stefan Peidli, Tessa D. Green, Ciyue Shen, et al. scPerturb: harmonized single-cell perturbation data. *Nature Methods*, 21:531–540, 2024. doi: 10.1038/s41592-023-02144-y.

[34] Yiqun Chen and James Zou. GenePT: A simple but effective foundation model for genes and cells built from ChatGPT. bioRxiv, 2023.

Supplementary material

Why Generate a Sentence? A Decision Language Model for Modeling Single-Cell Composition

References to Sections 1–6, Figures 1–3, Tables 1–5 and to cited works point to the main paper.

A Gene-level five-fold analysis (Norman)

This analysis was run with the Qwen3.5-0.8B backbone (CAD-P, the flat decoder of the population-level analysis) before the Gemma backbone was added. Pooled over five gene-level folds, the flat decoder ties the partial-additive baseline on pairs of seen genes (-0.017, CI -0.060 to $+0.021$, $n = 28$) and on pairs with one unseen gene ($+0.004$, CI -0.014 to $+0.022$, $n = 51$), and it beats ridge (Table S1, Figure S1). The split-half oracle lies 0.108 nats below the perturbed mean on unseen single genes, so headroom exists; descriptions and shuffled names do not help, and shuffled names are harmful ($+0.045$). A one-parameter target-induction baseline, which raises the unseen target gene's frequency, is better than every learned decoder on the 75 genes whose target is in the panel (-0.013, CI -0.031 to -0.004). On the test split, CAD-P (Qwen backbone) is 0.097 better than the perturbed mean and ties additive (-0.003, CI -0.031 to $+0.019$).

Paired NLL/UMI difference [95% CI]	unseen singles ($n = 84$)	pairs (seen genes) ($n = 28$)	pairs (one unseen gene) ($n = 51$)
to the perturbed mean			
additive (partial)	+0.000 [+0.000, +0.000]	−0.127 [−0.186, −0.077]	−0.076 [−0.111, −0.044]
ridge	**−0.002 [−0.010, +0.006]**	−0.033 [−0.053, −0.013]	−0.015 [−0.025, −0.006]
CAD-P, real gene names	+0.007 [−0.010, +0.023]	−0.144 [−0.197, −0.095]	−0.072 [−0.101, −0.046]
CAD-P, opaque ids	+0.015 [+0.002, +0.031]	−0.147 [−0.208, −0.099]	−0.066 [−0.090, −0.042]
CAD-P, shuffled names	+0.045 [+0.027, +0.063]	−0.155 [−0.208, −0.108]	−0.053 [−0.079, −0.027]
CAD-P + own description	+0.007 [−0.008, +0.022]	**−0.157 [−0.211, −0.112]**	**−0.084 [−0.112, −0.057]**
CAD-P + other gene's description	+0.018 [+0.003, +0.033]	−0.153 [−0.197, −0.114]	−0.072 [−0.102, −0.045]
split-half oracle (reference)	−0.108 [−0.147, −0.075]	−0.155 [−0.215, −0.105]	−0.151 [−0.188, −0.118]
between decoders			
real − opaque ids (primary)	−0.008 [−0.029, +0.011]	+0.003 [−0.019, +0.030]	−0.006 [−0.025, +0.013]
real − shuffled names	−0.038 [−0.061, −0.016]	+0.011 [+0.000, +0.025]	−0.019 [−0.046, +0.004]
description − other description	−0.011 [−0.022, −0.001]	−0.003 [−0.024, +0.012]	−0.012 [−0.022, −0.000]
description − real names	+0.000 [−0.010, +0.011]	−0.013 [−0.026, −0.002]	−0.012 [−0.024, −0.000]
CAD-P − additive	+0.007 [−0.010, +0.023]	−0.017 [−0.060, +0.021]	+0.004 [−0.014, +0.022]
CAD-P − ridge	+0.009 [−0.008, +0.025]	−0.111 [−0.167, −0.062]	−0.057 [−0.084, −0.033]

Table S1: Gene-level 5-fold analysis on Norman (CAD-P, Qwen3.5-0.8B backbone): pooled paired NLL differences (nats per UMI; negative favors the first term). Unit: perturbation.

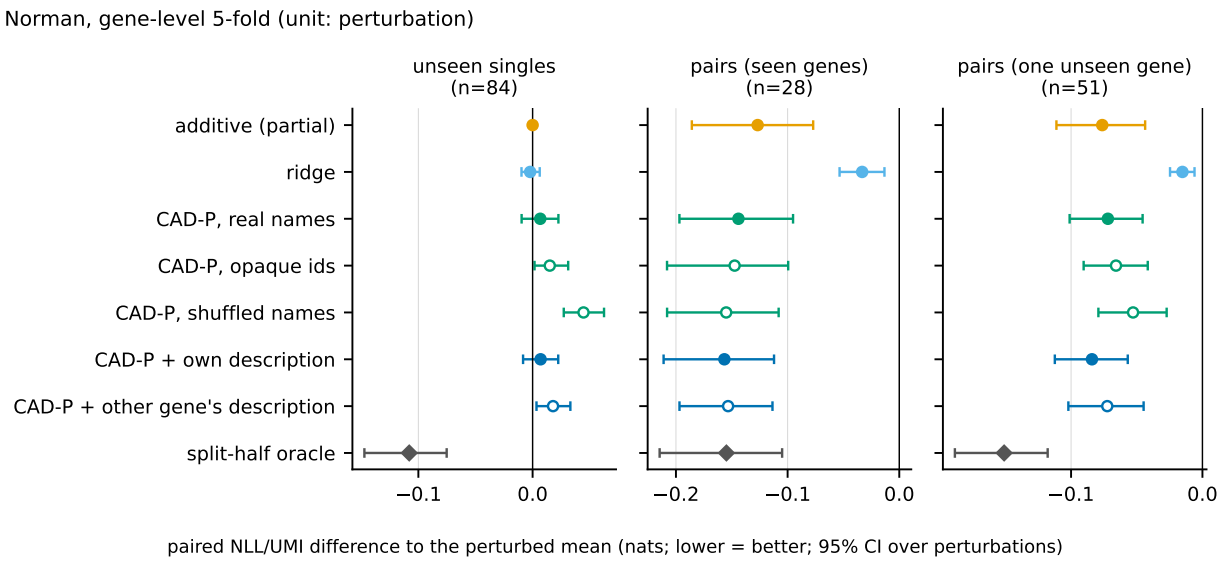


Figure S1: Gene-level 5-fold on Norman, population level: paired held-out NLL difference to the perturbed mean (bars: 95% bootstrap over perturbations; hollow: identity or knowledge controls; diamond: split-half oracle).

B Training details

Table S2 lists the training settings of the decision language models, and the paragraphs below it describe the scVI-based baselines.

Setting	Value
Optimizer, schedule	AdamW (weight decay 0), one-cycle schedule with 5% warm-up, gradient clipping at 1.0
Learning rates	2×10^{-4} (LoRA adapters), 10^{-3} (decision head, zero-initialized)
Batch size, budget	16; one epoch over 20,000 training cells drawn at random (1,250 steps)
LoRA	rank 16, $\alpha = 16$, dropout 0, all linear layers; rank 8 and 64 for the sensitivity run
Precision, attention	bf16; eager attention and gradient checkpointing for Gemma-2
Output vocabulary	the 2048 panel genes as classes; no new tokens, gene names are tokenized as plain text
Control-cell text	48 most abundant genes with 14 log-mass levels; 200 gene names for the rank variant
Seeds	test: Gemma and Qwen 0, 1, 2; validation: Gemma 3407 (default), 1, 2; Qwen 1, 2; variants of Appendix G: Qwen seed 0 (the seed changes the sampled training cells and the sampler)
Negative-binomial head	inverse dispersion $\log \theta_g(c) = \beta_g + v_g^\top h$, initialized at $\theta = 1$ and clipped to $[e^{-6}, e^{8}]$; loss: negative-binomial NLL of the raw panel counts with mean $N_i q_g$, per UMI
Low-rank head	$k = 16$ programs; amplitude $a(h) = \mathrm{softplus}(\cdot)$ initialized at 1; encoder of the target cell: $\log(1 + 10^4 x/N) \to 256$ (GELU) $\to 2k$; one reparameterized sample per cell; KL weight raised linearly from 0 to 1 over the first 30% of steps
C2S-Gemma baseline	C2S-Scale-Gemma-2-2B, LoRA $r = 16$ on all linear layers, lr 2×10^{-4}, one-cycle schedule, 20,000 cells, one epoch, batch 16 (micro-batches of 2); control and target as top-200 rank sentences; loss on the response only; sampling with top-k 50, top-p 0.95, up to 1024 new tokens
Evaluation	$M = 50$ held-out control cells per perturbation, the same for every method (earlier runs: drawn separately per method); 300 generated cells per unit
Hardware, time	one A100 40 GB; run alone, Qwen takes 6 to 8.5 min and Gemma 9.5 to 24 min per run (90 to 100 min with the 200-gene rank text; wall-clock was longer when jobs shared the GPU); Gemma peak memory 8.7 GB

Table S2: Training settings of the decision language models.

scVI and scANVI. Both are trained on training cells of the panel with a negative-binomial likelihood, 30 latent dimensions, and 60 epochs (scANVI: 15 further epochs with cell-type labels). The frequency vector of a cell type is the mean decoder output over up to 3000 training cells of that type.

scGen-style baseline. An scVI model is trained on all training cells. The latent shift of a single gene is the mean latent state of its perturbed cells minus that of the control cells, the shift of a pair is the sum of the shifts of its known

genes, and a perturbation with no known gene uses the mean shift of all training perturbations. Each held-out control cell is encoded, shifted, and decoded with library size one.

C Negative-binomial evaluation

For a held-out cell i of a unit with library size s_i and model frequency q, the count of gene g is scored as $\mathrm{NB}(x_{ig} \mid \mu = s_i q_g, \phi_g)$ with variance $\mu + \phi_g \mu^2$. The dispersions ϕ_g come from a method-of-moments fit on library-normalized training counts with the within-unit variance pooled over units (median $\phi = 26.5$ for PBMC and 9.0 for Norman, clipped to $[10^{-3}, 10^{2}]$). The score is the negative log-likelihood per UMI, which is on a different scale from the multinomial score because it includes the zeros, so only differences are compared (Table S3). It uses the pooled frequency of a unit, so it does not reward conditioning on a control cell.

Task	Model	ΔNB [95% CI]	Δmultinomial
PBMC (vs. cell-type mean)	global mean	$+0.302$ [$+0.252$, $+0.352$]	$+1.125$
	scVI	$+0.004$ [$+0.001$, $+0.007$]	$+0.005$
	scANVI	$+0.002$ [$+0.000$, $+0.005$]	$+0.004$
	CAD (Gemma)	$+0.004$ [$+0.002$, $+0.006$]	$+0.017$
	CAD (Qwen)	$+0.005$ [$+0.002$, $+0.009$]	$+0.030$
Norman (vs. perturbed mean)	additive	-0.014 [-0.023, -0.004]	-0.094
	ridge	-0.008 [-0.010, -0.006]	-0.024
	scGen-style	-0.014 [-0.021, -0.008]	-0.085
	CAD (Gemma, no control)	-0.021 [-0.027, -0.015]	-0.104
	CAD (Gemma, control)	-0.017 [-0.023, -0.012]	-0.091
	CAD (Qwen, no control)	-0.019 [-0.025, -0.013]	-0.092
	CAD (Qwen, control)	-0.015 [-0.020, -0.010]	-0.078

Table S3: Negative-binomial and multinomial NLL per UMI relative to the reference (test split, seed 0; earlier runs, with control cells not matched across methods; negative favors the model; paired bootstrap over cell types or perturbations).

On PBMC the negative-binomial score separates the decoders slightly more than the multinomial score does: CAD is reliably, but marginally (0.004 nats), worse than the cell-type mean. On Norman the ordering of the methods is the same as under the multinomial score.

D All worked examples: prompts and decoded distributions

Figures S2 and S3 show, for every use case, the exact prompt that the decision language model reads and the distribution it returns, next to the held-out ground truth. The examples are not hand-picked. In each case, the shown unit is the held-out cell type or perturbation whose paired NLL difference between CellAsDistribution and the case's reference is the median over the units of that case, so half of the units look better and half look worse. The cases are:

- conditional generation on unseen PBMC donors, against the cell-type mean;
- perturbation response without a control cell, one example per stratum of the gene-level split (unseen single gene, pair with one unseen gene, pair of two seen genes), against the additive baseline;
- perturbation response with a control cell, for a pair of two seen genes, once with the control cell both written as mass-level text in the prompt and used as the residual prior (the prompt shows a real serialized control cell), and once with the control cell used only as the residual prior;
- zero-shot cell types (fold 0, so the median of its 4 held-out types), against the mean of the nearest training type in the Cell Ontology.

The ground truth is the pooled frequency of all held-out real cells of the unit. The decision head outputs one probability per panel gene, always in the same panel order, so the distributions are drawn on a *fixed gene axis*: the 25 panel genes with the highest mean frequency in training cells, in panel order, the same for every unit of a dataset; only the heights change from unit to unit and from cell to cell. For a control-cell model, the large dot is the mean over the $M = 50$ matched control cells and the hollow dots are the distributions decoded for three individual control cells, which differ

because each control cell enters as its own log-prior; the prompt shown is the one built from the first control cell. For perturbations, the right panel compares the log fold change over the held-out control mean on the unit's top-20 differentially expressed genes (largest $|t|$ of a Welch test between its real cells and the held-out control cells, computed on real cells only). The selected units and the numbers quoted for them are:

(a) *PBMC, conditional generation*: naive thymus-derived CD4-positive, alpha-beta T cell, the median of 24 units (NLL difference to the cell-type mean: $+0.011$ nats per UMI).

(b) *K562, no control cell, unseen single gene*: MAML2, the median of 21 units (NLL difference to the additive baseline: -0.015 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.33 for CAD and 0.19 for the reference.

(c) *K562, no control cell, pair with one unseen gene*: FOXF1+FOXL2, the median of 29 units (NLL difference to the additive baseline: $+0.011$ nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.92 for CAD and 0.96 for the reference.

(d) *K562, no control cell, pair of two seen genes*: DUSP9+SNAI1, the median of 26 units (NLL difference to the additive baseline: -0.012 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.91 for CAD and 0.95 for the reference.

(e) *K562, control cell as text and residual, pair of two seen genes*: ETS2+CNN1, the median of 26 units (NLL difference to the additive baseline: $+0.006$ nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.91 for CAD and 0.98 for the reference.

(f) *K562, control cell as residual only, pair of two seen genes*: FOXA1+FOXL2, the median of 26 units (NLL difference to the additive baseline: -0.004 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.87 for CAD and 0.83 for the reference.

(g) *PBMC, zero-shot cell type (fold 0)*: naive thymus-derived CD8-positive, alpha-beta T cell, the median of 4 units (NLL difference to the nearest Cell Ontology type: -0.728 nats per UMI).

E Transformed training targets (log1p, square root)

Table S4 (validation, 19 cell types, Qwen3.5-0.8B backbone) shows that the transformation trades likelihood for sample quality on PBMC. Frequency gives the best NLL, and smoothing is worse on every metric ($+0.172$ nats, 95% CI $+0.144$ to $+0.196$; MMD $+0.074$). Log1p and square-root targets are $+0.144$ and $+0.118$ nats worse in NLL, yet their MMD is lower (-0.057, CI -0.107 to -0.016, and -0.057, CI -0.109 to -0.014) with unchanged real-versus-generated AUC. A flatter target moves mass to rarely expressed genes, which lowers the likelihood of raw counts while placing generated cells closer to real ones in PCA space.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑	Type acc. ↑
frequency	**5.629**	**5.629**	0.1508	0.963	0.947	**0.887**
smoothed	5.801 (+0.172 [+0.144, +0.196])	5.801	0.2245	0.994	0.923	0.761
log1p	5.773 (+0.144 [+0.097, +0.199])	5.773	**0.0934**	**0.958**	**0.955**	0.854
sqrt	5.747 (+0.118 [+0.087, +0.152])	5.747	0.0939	0.962	0.955	0.835

Table S4: Human PBMC (Hao et al.), validation split: 19 held-out cell types; CAD with the Qwen3.5-0.8B backbone and four target transformations. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to frequency [95% bootstrap over cell types]). MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation), type acc.: a classifier trained on real cells recovers the requested type. The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

For perturbation response, the trade-off does not appear (Gemma backbone, test, 76 perturbations; Appendix K). Relative to the frequency target, log1p is 0.288 nats worse in NLL (CI $+0.249$ to $+0.333$) and square root 0.250 worse (CI $+0.219$ to $+0.282$), and both also have a higher MMD (0.094 and 0.087 against 0.039) and a higher real-versus-generated AUC (0.875 and 0.871 against 0.811). We therefore keep frequency as the default, and we report the sample-quality advantage of flattened targets as specific to the conditional PBMC task in our experiments.

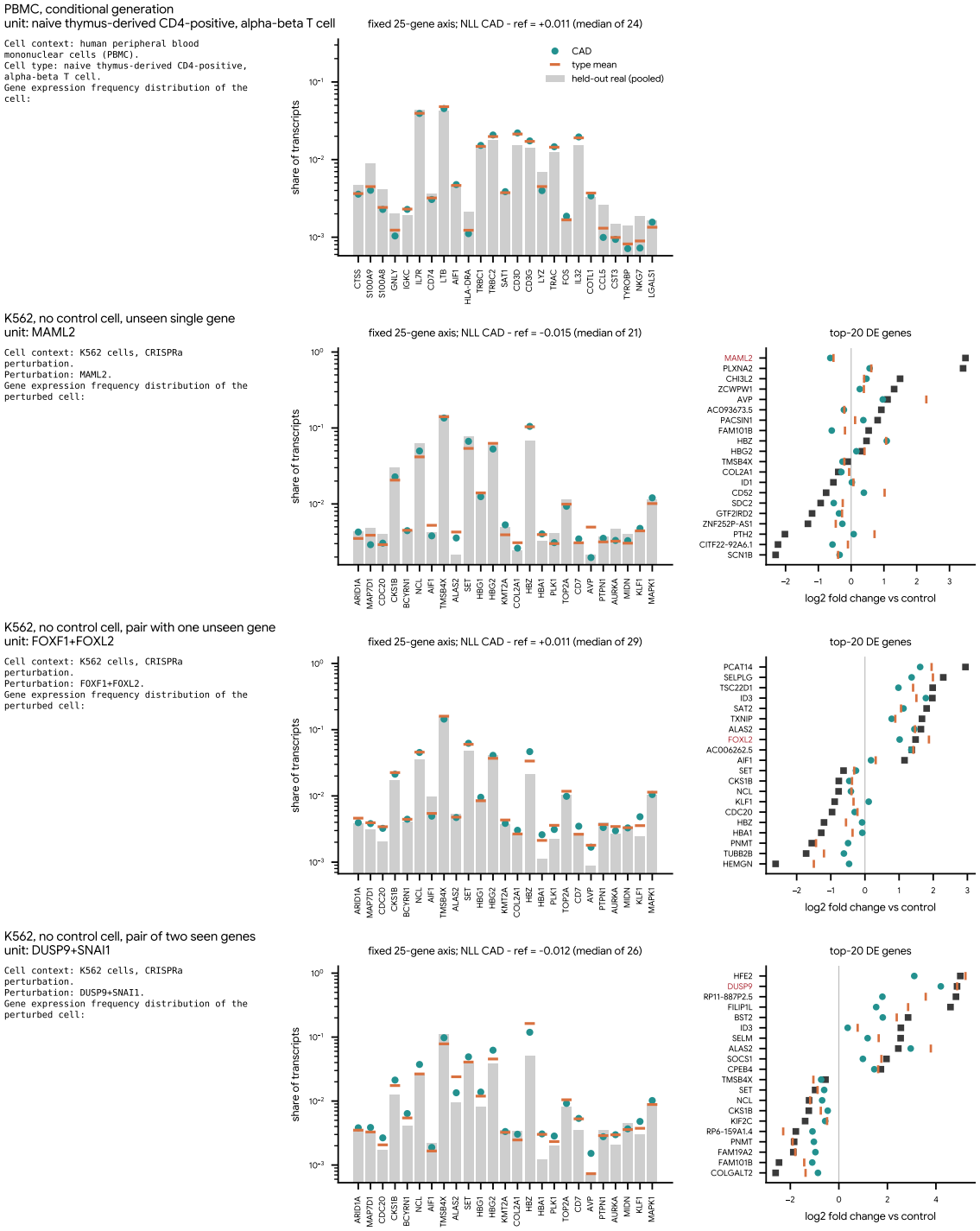


Figure S2: Prompts and decoded distributions, one median example per use case (test split). Left: the prompt given to CellAsDistribution. Middle: the decoded distribution on a fixed axis of 25 genes (chosen once from training cells, shown in panel order, identical in every row of a dataset; log scale) for the held-out real cells (grey bars), CellAsDistribution (teal dots), and the reference (orange ticks). Right, perturbations only: log fold change over the held-out control mean on the unit's top-20 DE genes for real cells, CellAsDistribution, and the reference. Shown here: conditional generation and the three strata without a control cell; the remaining cases are in Figure S3.

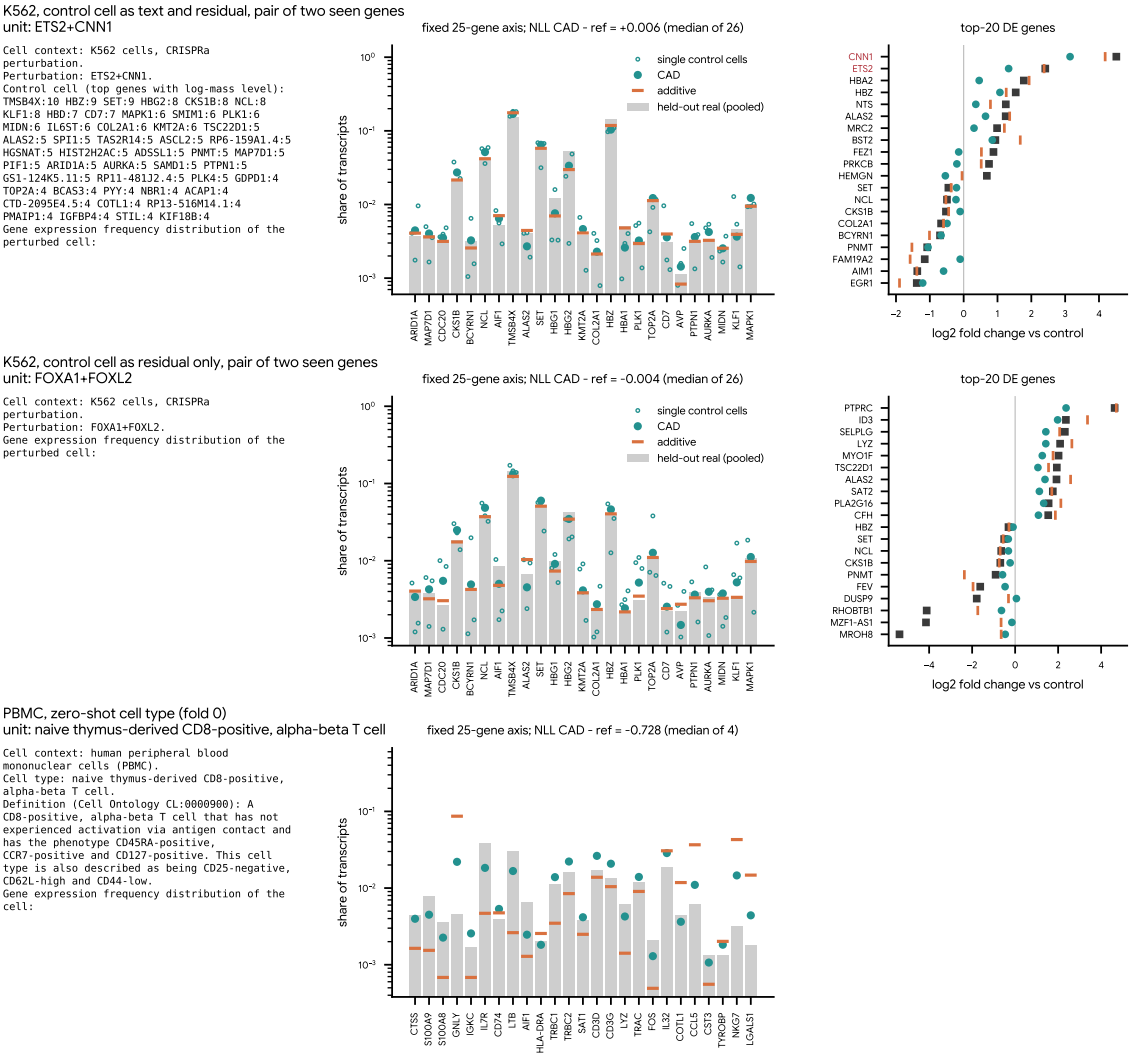


Figure S3: Prompts and decoded distributions (continued from Figure S2): perturbation response with the control cell as text and residual, and as residual only (hollow dots: distributions decoded for three individual control cells on the same gene axis), and zero-shot cell types.

F UMAP views of generated cells

Figures S4 and S5 show UMAP projections of held-out real and generated cells for the Norman K562 perturbation task and the human PBMC conditional generation task.

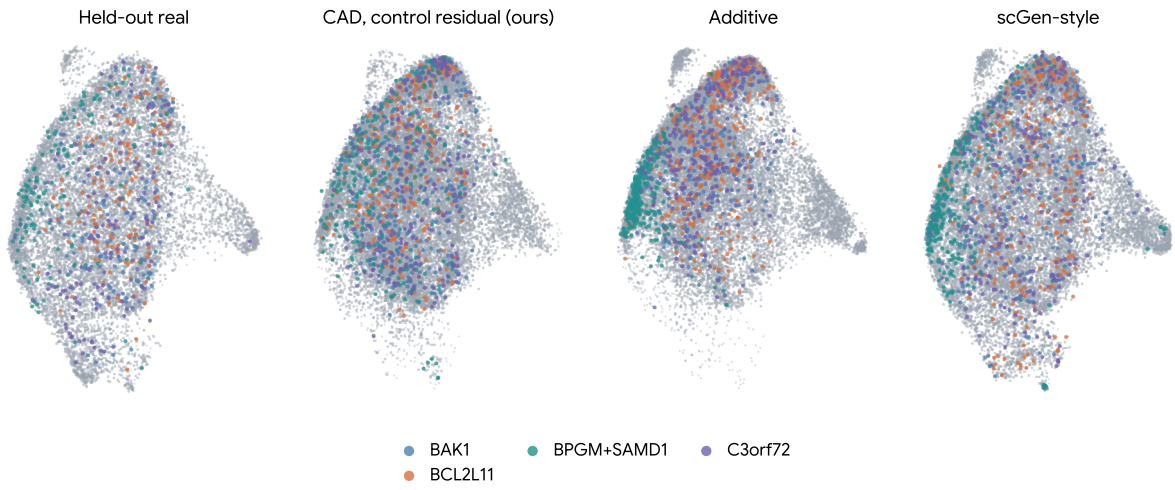


Figure S4: UMAP of held-out real perturbed K562 cells (test split) and of cells generated from the matched control cells by CellAsDistribution with the control-cell residual (Gemma, seed 0), by the additive baseline, and by the scGen-style latent shift, all drawn with the shared multinomial sampler. PCA and UMAP are fit on real cells only, and generated cells are projected. Colors: the four most frequent held-out perturbations; grey: the others.

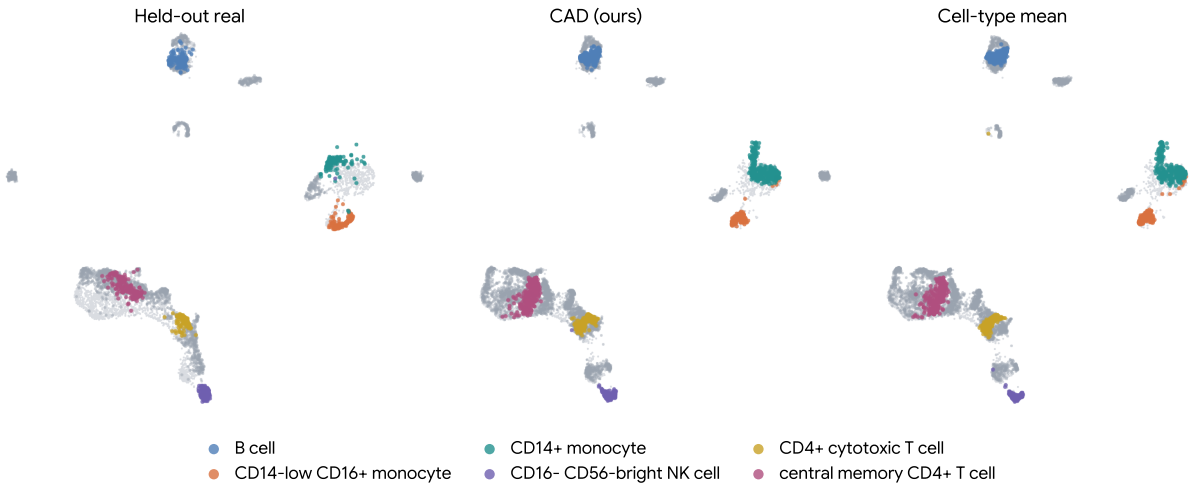


Figure S5: UMAP of held-out real PBMC cells (unseen donors, test split) and of cells generated by CellAsDistribution (Gemma, seed 0) and by the cell-type mean with the same sampler. PCA and UMAP are fit on real cells only, and generated cells are projected. Colors: the six most frequent cell types; grey: the others.

G Pre-registered decision rules and the test-split record

Beyond the main configurations, every variant was first run on validation (Qwen3.5-0.8B backbone, seed 0, matched control cells, comparators re-run with the same control cells) and promoted to the test split only if, on validation, its

paired 95% interval against its comparator excluded zero in its favor on at least one metric fixed in advance, and no such metric was worse with an interval excluding zero; at most three new configurations were promoted. Promoted configurations were then run once per seed (0, 1, 2) on test with the Gemma backbone. The rules and decisions were written to the repository before the test runs.

Variant	Comparator	Primary metric(s)	Decision
Control residual, no control text	control residual with mass text	per-cell NLL	beat comparator; test, 3 seeds
Control text, no residual	control residual with mass text	per-cell NLL	worse; test, 1 seed (ablation)
Contrast control text	control residual with mass text	per-cell NLL, MMD, r_Δ	promoted
Gene summaries in the prompt	other gene's summary; opaque ids	NLL, r_Δ on unseen singles	not promoted
UMI-weighted loss	same model, equal weights	NLL, r_Δ, r_{DE}	not promoted
Negative-binomial head	softmax head	NLL, r_Δ, r_{DE}	promoted (Norman only)
Low-rank head	softmax head	per-cell NLL, MMD, AUC	promoted (no control cell)
Zero-shot cell types	nearest Cell Ontology type	NLL	passed (gate G3)
C2S-Gemma mixing weight	—	validation NLL	0.9 (grid maximum)

Table S5: Validation decisions taken before the test runs.

Four analyses were added after the test results had been seen, and are labeled as such where they appear: the R^2 of the rank reconstruction (Section 5.1), the control-cell residual without a language model (Section 5.4; a first version that divided by the pooled control frequency instead of the mean prior was not a fair analogue and was discarded), the name-overlap and Cell Ontology parent baselines for zero-shot types (Section 5.5), and the decoding-speed benchmark (Section 5.2). Each can only weaken our claims.

Deviations, all disclosed: a 128-gene mass-level control text was not evaluated because it repeatedly ran out of GPU memory; three Qwen control-residual test runs ran out of memory during the test lanes and were re-run alone with the same configuration; and the zero-shot gate was applied on the 8 types of folds 0 and 1 because three validation runs of folds 2–4 had run out of memory, which were then re-run (on all 19 types the gate gives the same decision, -0.476 nats per UMI, CI -0.971 to -0.123).

Revision 3 (written before its new runs). Two controls for the language model (a frozen backbone with an MLP head, and a decoder without a language model; Section 5.2) were specified, with their comparators, metrics, and decision rule, in `PREREGISTRATION_rev3.md` (under `experiments/`) before they were run. As controls rather than candidate improvements, they were run on validation (Qwen, seed 0) and then on test (Gemma, seeds 0–2) whatever their validation result. Four further analyses of saved outputs were added after the test results had been seen: the downstream and target metrics with the perturbed genes excluded and the additive-plus-target-induction baseline (Section 5.3); the learned rank-to-frequency maps, including the re-decoding of the C2S-Gemma sentences (Section 5.1); and the co-expression comparison (Appendix I). They use no new training on test data. Without exclusion, the downstream code reproduces the earlier r_{DE} values exactly.

Revision 4 (written before its runs). Four further analyses were specified in `PREREGISTRATION_rev4.md` (under `experiments/`) before they were run and are reported whatever their outcome: zero-shot cell types with the C2S-Scale-Gemma backbone (the same folds, prompts, and settings as the Qwen runs); the no-language-model decoder with frozen GenePT or co-expression gene embeddings (test, seeds 0–2); the add-on panel of Appendix J; and CellAsDistribution with the target prior, applied to its saved output. All were added after the test results had been seen and none was promoted. The reporting rules fixed in advance were: Gemma counts as reading names better than string matching only if its interval against name overlap excludes zero (it does); GenePT counts as transfer only if it beats learned embeddings on unseen single genes with an interval excluding zero (it is worse).

H Negative-binomial and low-rank heads

Negative-binomial head. The head adds a per-gene inverse dispersion $\theta_g(c)$, $\log \theta_g(c) = \beta_g + v_g^\top h$, and is trained on the negative-binomial likelihood of the counts with mean $N_i q_g$, so high-mass genes no longer dominate the gradient. On the Norman test split (no control cell, against the softmax head) it raises r_Δ by 0.020 (CI 0.012 to 0.029), and by 0.049 on unseen single genes (CI 0.034 to 0.067), at a small cost in pooled multinomial NLL ($+0.015$, CI $+0.006$ to $+0.028$). On PBMC it did not help on validation and was not promoted.

Low-rank logistic-normal head. The logits become $z = Wh + \log b + U\,\mathrm{diag}(a(h))\,\epsilon$ with $\epsilon \sim \mathcal{N}(0, I_{16})$, so $q(c, z)$ varies from cell to cell along 16 learned gene programs; training uses a variational bound with an encoder of the target cell, and generation draws ϵ from the prior. Without a control cell it lowers the cell NLL by 0.089 (CI 0.080 to 0.098) on Norman, reaching the scGen-style model (-0.009, CI -0.019 to $+0.002$), and on PBMC from 5.623 to 5.552. The pooled NLL of the mean over draws is worse ($+0.062$, CI $+0.040$ to $+0.087$) and r_Δ lower (-0.067). The cell NLL is estimated from 50 (Norman) or 300 (PBMC) prior draws; this log-mean-exp estimator is biased upwards.

I Co-expression of generated cells

For each test perturbation with at least 100 real cells, on the 100 panel genes with the largest variance of log-normalized expression in training cells, we compared the gene–gene Pearson correlations across real cells with those across the method's generated cells (multinomial sampler), and the total variance and the share of variance on the first principal component (Table S6; post hoc). With a median of about 216 panel UMIs per cell, correlations are dominated by counting noise, and every generator is within 0.005 of the others in correlation error. Resampling real cells of training perturbations is further from the held-out cells (0.093), so the generated cells reproduce perturbation-specific co-variation at least as well as real cells of other conditions. The low-rank head and the scGen-style shift, which model variation between cells explicitly, are slightly closer than the flat decoders, and the additive baseline under-disperses (variance ratio 0.94).

Generator	correlation error ↓	variance ratio (→ 1)	PC1-share error ↓
Additive	0.078	0.94	0.027
Additive, control residual (no LM)	0.078	**1.01**	0.027
CellAsDistribution, control residual	0.078	1.04	0.028
CellAsDistribution, low-rank head	**0.074**	1.04	0.021
CellAsDistribution, no control cell	0.079	**0.99**	0.026
Training-cell resampling	0.093	1.02	0.018
scGen-style	0.075	**1.01**	**0.017**

Table S6: Co-expression of generated cells, Norman test split (71 perturbations with at least 100 cells; 100 training-selected variable genes; seed 0; post hoc). Correlation error: mean absolute difference between gene–gene correlations across real and generated cells; variance ratio: total variance generated/real (1 is ideal). Best value of each column in bold; arrows give the better direction.

J An add-on panel with every perturbed gene

The main panel admits only training targets by force, so 17 of the 21 unseen single targets are not scored. We rebuilt the Norman panel with every perturbed gene forced in (`preprocess.py -force-all-targets`; only gene identities are used, and the remaining genes are chosen by training-cell dispersion as before), checked that the cell-level splits are identical, and re-ran the additive and target-induction baselines, the three no-language-model decoders, and CellAsDistribution (Gemma, no control cell) once on test (Table S7; post hoc). Three of the 105 perturbed genes are not in the dataset's gene list, which leaves 19 unseen single genes with their target in the panel. Every learned model predicts almost no induction of an unseen target; the one-factor target-induction baseline halves the error. Likelihood orderings match the main panel.

Model	ΔNLL to additive ↓			target error ↓			target sign ↑
	single	1 unseen	2 seen	single	1 unseen	2 seen	single
Additive	0	0	0	1.81	0.78	**0.38**	0.26
Perturbed mean	+0.000	+0.114	+0.141	1.81	1.49	1.61	0.26
Additive + target induction	−0.006	**−0.003**	+0.000	**0.93**	**0.45**	**0.38**	**0.95**
No LM (learned)	**−0.017**	+0.006	**−0.036**	1.73	0.85	0.40	0.32
No LM (GenePT)	+0.004	+0.065	−0.014	1.72	1.17	0.76	0.32
No LM (co-expression)	+0.029	+0.072	−0.020	1.60	0.88	0.42	0.42
CellAsDistribution, no control cell	−0.007	+0.010	−0.028	1.73	0.91	0.61	0.26

Table S7: Add-on gene panel with every perturbed gene forced in (21 unseen single genes, 19 of them with their target in the panel; same splits as the main panel, verified cell by cell; Norman test split, seed 0, C2S-Scale-Gemma-2-2B for CellAsDistribution; post hoc). ΔNLL: mean paired difference to additive; target error: mean absolute error of the predicted log fold change of the perturbed gene(s); target sign: fraction of unseen single targets predicted to go up. Best value of each column in bold; arrows give the better direction.

K Additional tables (all methods, validation, sensitivity)

The tables in this section and in Appendix E come from the earlier runs, in which the control cells were drawn separately for each method; the main-text tables use matched control cells. Table S8 reports all methods on the Norman test split. Table S9 (test) compares the target transformations for perturbation response with the Gemma backbone, and Table S10 (test) compares the two control-cell formats. Tables S11 and S12 repeat the comparison with the other generators on the validation split, on which all decisions were made; Table S13 shows the LoRA-rank sensitivity on Norman (validation).

Model	NLL ↓	ΔNLL to additive ↓	cell NLL ↓	MMD ↓	AUC ↓	r_Δ ↑	r_{DE} ↑
Control mean	5.352	+0.158 [+.108, +.215]	5.352	0.085	0.866	0.23	0.41
Perturbed mean	5.289	+0.094 [+.058, +.136]	5.289	0.073	0.903	0.12	0.58
Additive (reference)	5.195	0	5.195	0.052	0.855	0.44	**0.77**
Additive + target induction	5.194	−0.001 [−.003, +.000]	5.194	0.051	0.857	0.44	**0.77**
Ridge on gene embeddings	5.265	+0.070 [+.031, +.114]	5.265	0.057	0.849	0.46	0.68
scGen-style	5.201	+0.006 [−.014, +.022]	5.105	**0.029**	**0.769**	0.51	0.72
GEARS	5.755	+0.561 [+.518, +.603]	5.755	0.198	0.980	0.14	0.49
C2S, autoregressive	5.288	+0.094 [+.059, +.133]	5.279	0.075	0.906	0.10	0.56
Additive, control residual	5.199	+0.005 [−.001, +.011]	5.134	0.038	0.815	0.44	0.76
CellAsDistribution, no control cell	**5.186**	**−0.009 [−.037, +.010]**	5.186	0.039	0.812	0.50	**0.77**
CellAsDistribution, control residual	5.197	+0.002 [−.025, +.022]	5.129	0.030	0.785	0.46	0.72
CellAsDistribution, NB head	5.201	+0.006 [−.021, +.028]	5.201	0.036	0.797	**0.52**	**0.77**
CellAsDistribution, low-rank head	5.247	+0.053 [+.024, +.084]	**5.096**	0.040	0.791	0.43	0.73
CellAsDistribution, opaque gene ids	5.200	+0.005 [−.024, +.025]	5.200	0.048	0.845	0.43	0.69

Table S8: All methods, Norman test split (76 perturbations, seed 0, matched control cells). r_Δ: Pearson correlation of predicted and real change relative to the training perturbed centroid over all panel genes except the perturbed ones; r_{DE} as in Table 2. GEARS and control-mean rows: perturbed genes not excluded (earlier run). Best value of each column in bold; arrows give the better direction.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
frequency	**5.186**	**5.186**	**0.0393**	**0.811**	**0.957**
log1p	5.474 (+0.288 [+0.249, +0.333])	5.474	0.0941	0.875	0.938
sqrt	5.435 (+0.250 [+0.219, +0.282])	5.435	0.0869	0.871	0.940

Table S9: Norman K562, test split: 76 held-out perturbations; CAD (Gemma), no control cell, with three target transformations. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to frequency [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
mass text	5.199	5.173	**0.0371**	0.816	**0.955**
rank text	**5.192 (−0.007 [−0.012, +0.000])**	**5.159**	0.0375	**0.809**	0.954

Table S10: Norman K562, test split: 76 held-out perturbations; CAD (Gemma), control residual, with the control cell given as mass levels or as a 200-gene rank sentence. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to mass text [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑	Type acc. ↑
Global mean	6.700 (+1.073 [+0.892, +1.283])	6.700	0.8462	1.000	0.565	0.048
Cell-type mean	**5.627**	**5.627**	0.1688	0.983	0.942	**0.928**
scVI	5.634 (+0.007 [−0.005, +0.018])	5.634	0.1576	0.968	0.946	0.906
scANVI	5.630 (+0.003 [−0.007, +0.012])	5.630	0.1494	0.967	0.947	0.878
CAD (Gemma)	5.628 (+0.001 [−0.007, +0.008])	5.628	**0.1440**	0.972	**0.948**	0.887
CAD (Qwen), seed 1	5.632 (+0.005 [−0.003, +0.011])	5.632	0.1464	**0.963**	0.947	0.900

Table S11: Human PBMC (Hao et al.), validation split: 19 held-out cell types; CAD rows: first seed unless stated; baselines: one run each. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to Cell-type mean [95% bootstrap over cell types]). MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation), type acc.: a classifier trained on real cells recovers the requested type. The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
Perturbed mean	5.314	5.314	0.0779	0.915	0.920
Additive	5.248 (−0.066 [−0.096, −0.040])	5.248	0.0459	0.838	0.952
Ridge	5.306 (−0.008 [−0.021, +0.004])	5.306	0.0670	0.874	0.936
scGen-style	5.271 (−0.043 [−0.072, −0.017])	**5.162**	**0.0300**	**0.774**	0.955
CAD (Gemma), no control cell	**5.235 (−0.079 [−0.105, −0.059])**	5.235	0.0360	0.799	**0.959**
CAD (Gemma), control residual, mass text	5.246 (−0.068 [−0.094, −0.046])	5.221	0.0393	0.824	0.952
CAD (Gemma), control residual, rank text	5.240 (−0.075 [−0.099, −0.054])	5.209	0.0370	0.821	0.954
CAD (Qwen), no control cell, seed 1	5.239 (−0.076 [−0.102, −0.055])	5.239	0.0400	0.823	0.955

Table S12: Norman K562, validation split: 48 held-out perturbations; CAD rows: first seed unless stated; baselines: one run each. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to Perturbed mean [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
rank 16	**5.235**	**5.235**	**0.0360**	**0.799**	**0.959**
rank 8	5.237 (+0.002 [−0.002, +0.007])	5.237	0.0365	0.812	0.959
rank 64	5.237 (+0.002 [−0.002, +0.007])	5.237	0.0386	0.809	0.957

Table S13: Norman K562, validation split: 48 held-out perturbations; CAD (Gemma), no control cell, with LoRA rank 16, 8 and 64. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to rank 16 [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

L Proofs (Proposition 1 and the hierarchical head)

Proposition 1. Part (ii) is P2 below. (i) $x = Np$ recovers the counts from (N, p), and (N, p) is a function of x. Under $\mathrm{Mult}(N, q)$, $\log P(x) = \log \frac{N!}{\prod_g x_g!} + \sum_g x_g \log q_g = \text{const} + N \sum_g p_g \log q_g$, which by Gibbs' inequality is maximized over $q \in \Delta$ at $q = p$. (iii, rank sentences) For any x with at least two distinct nonzero counts, x and $2x$ have the same ordering and hence the same sentence, and so do x and any count vector obtained by a strictly increasing change of the counts. (iii, transformed vectors) x and $2x$ have the same binarization. For $f(t) = \log(1+t)$ or $f(t) = \sqrt{t}$, the renormalized vector is $p'_g = f(x_g)/\sum_h f(x_h)$, and $p' = p$ requires $f(x_g)/x_g$ to be equal for all genes with $x_g > 0$; because $f(t)/t$ is strictly decreasing for $t > 0$, this holds only when all nonzero counts are equal.

Throughout, Δ is the simplex over the G panel genes and modules $S_1, \ldots, S_M$ partition the genes; P1, P3, and P4 concern the hierarchical head of Appendix M, and P2 concerns the NLL used throughout.

P1 (normalization and exact factorization). For any $q^{\mathrm{root}} \in \Delta^M$ and $q^m \in \Delta^{|S_m|}$, $q(g) = q^{\mathrm{root}}(m(g))\, q^{m(g)}(g)$ lies in Δ, and for counts X with module totals n_m,

$$-\sum_g X_g \log q(g) = -\sum_m n_m \log q^{\mathrm{root}}(m) - \sum_m \sum_{g \in S_m} X_g \log q^m(g).$$

Every q in the interior of Δ factors uniquely this way, so the hierarchical head is a *re-parametrization* of the flat head: it changes what the network must output, not the family of distributions it can represent.

P2 (the metric is proper). If the counts X of a perturbation's cells satisfy $\mathbb{E}[X_g/\sum_h X_h] = \pi_g$ given the total, the expected NLL per UMI of q is $H(\pi) + \mathrm{KL}(\pi \,\|\, q)$ and is uniquely minimized at $q = \pi$. Differences in expected NLL between two decoders therefore equal differences in $\mathrm{KL}(\pi \| q)$.

P3 (tilt identity and its population minimizer). For $f, \tilde{p}$ in the interior of Δ, $q_g \propto f_g t_g$ equals $\tilde{p}_g$ exactly. If a context is paired with bags b having tilts t_b, the minimizer of the expected soft cross-entropy is $\mathbb{E}[t_b]$, so the implied decoder is $q_g \propto \mathbb{E}_b[\tilde{p}_{b,g}/Z_b]$: a $1/Z_b$-weighted average of bag frequencies, which equals the plain average whenever Z_b is constant within a context. The target is the shrunk $\tilde{p}$, so $\|\tilde{p} - p\|_1 \le \frac{\lambda}{n+\lambda}\|f - p\|_1 \le 2\frac{\lambda}{n+\lambda}$ ($\approx 1\%$ for $n \approx 2\times 10^5$ UMIs per bag).

P4 (sampled-leaf loss). Let $T_m = \sum_{g \in S_m} t_g$. With modules $A = (m_1, \ldots, m_R)$ drawn i.i.d. from T, $\mathrm{CE}(T, r^{\mathrm{root}}) + \mathbb{E}\big[\frac{1}{R}\sum_r \mathrm{CE}(t|_{m_r}, r^{m_r})\big] = \mathrm{CE}(t, r)$, the full tree cross-entropy. *Our implementation samples without replacement and the trainer averages questions with equal weight, so its record loss is a weighted surrogate* ($1/(R{+}1)$ per question), not this unbiased estimator; the two share their minimizer only when every conditional is exactly representable (the well-specified regime).

P1. $\sum_g q(g) = \sum_m q^{\mathrm{root}}(m) \sum_{g \in S_m} q^m(g) = \sum_m q^{\mathrm{root}}(m) = 1$ and every factor is nonnegative. The loss identity follows from $\log q(g) = \log q^{\mathrm{root}}(m(g)) + \log q^{m(g)}(g)$ and $\sum_{g \in S_m} X_g = n_m$. Conversely, for q in the interior set $q^{\mathrm{root}}(m) = \sum_{g \in S_m} q(g)$ and $q^m(g) = q(g)/q^{\mathrm{root}}(m)$; these are the only factors that reproduce q because $q^{\mathrm{root}}(m) = \sum_{g \in S_m} q(g)$ is forced by summing.

P2. Conditional on the total N, $\mathbb{E}[-\sum_g X_g \log q_g / N] = -\sum_g \pi_g \log q_g = H(\pi) + \mathrm{KL}(\pi \| q)$, and $\mathrm{KL} \ge 0$ with equality if and only if $q = \pi$ (Gibbs' inequality). Two decoders therefore differ in expected NLL by

$\mathrm{KL}(\pi\|q_A) - \mathrm{KL}(\pi\|q_B)$. Per-UMI scoring weights each perturbation's cells by their UMIs; the paired bootstrap treats the perturbation as the unit.

P3. $f_g t_g = \tilde{p}_g/Z$, so $\sum_h f_h t_h = 1/Z$ and $f_g t_g / \sum_h f_h t_h = \tilde{p}_g$. For targets t_b drawn with context c,

$$\mathbb{E}_b[\mathrm{CE}(t_b, r)] = H(\mathbb{E}t_b) + \mathrm{KL}(\mathbb{E}t_b\|r) + \text{const}$$

is minimized by $r = \mathbb{E}[t_b]$, so $q_g \propto f_g \mathbb{E}[t_{b,g}] = \mathbb{E}[\tilde{p}_{b,g}/Z_b]$, a $1/Z_b$-weighted mean of $\tilde{p}_b$. With $\tilde{p} = (1-w)p + wf$ and $w = \lambda/(n+\lambda)$, $\|\tilde{p} - p\|_1 = w\|f - p\|_1 \le 2w$.

P4. The chain rule gives $\mathrm{CE}(t, r) = \mathrm{CE}(T, r^{\mathrm{root}}) + \sum_m T_m \,\mathrm{CE}(t|_m, r^m)$ with $t|_m = t_{S_m}/T_m$. For $m_1, \ldots, m_R$ i.i.d. from T, $\mathbb{E}[\mathrm{CE}(t|_{m_r}, r^{m_r})] = \sum_m T_m \mathrm{CE}(t|_m, r^m)$ for each r, which gives the claim. Sampling without replacement changes the inclusion probabilities away from T_m, and averaging over the $R{+}1$ answered questions gives each question weight $1/(R{+}1)$ instead of $(1, \frac{1}{R}, \ldots, \frac{1}{R})$; if every conditional is representable, each term is minimized at the true conditional and the minimizer is unchanged.

M Hierarchical decision head (ablation)

The hierarchical variant (CAD-N/CAD-R) answers one 64-option module question and four 32-option gene questions through the Unsloth decision head, trained on a *tilt* target (the ratio of the perturbed to the control frequency, renormalized, with the control prior f multiplied back at generation). On the corrected Norman split, trained on absolute frequencies it is $+0.161$ [$+0.150$, $+0.174$] nats worse than the perturbed mean (control-state NLL 5.490 against 5.295 for the control frequencies); the tilt repairs the fit (control 5.300; tilt minus no-tilt -0.138 [-0.157, -0.119]) but the tilted head remains $+0.023$ [$+0.009$, $+0.038$] worse than the perturbed mean and 0.100 nats worse than the flat head. The biological partition beats a random one by 0.019 ($[-0.035, -0.002]$). We therefore use the flat head in the main text. Because the hierarchical head is a re-parametrization of the flat one (P1), it cannot represent anything the flat head cannot; its deficit is a training effect, and the weighted, without-replacement surrogate of P4, which shares the exact tree cross-entropy's minimizer only when every conditional is representable, may account for part of it. Training it on the exact factorized loss of P1 is cheap at $G = 2048$ and was not done.